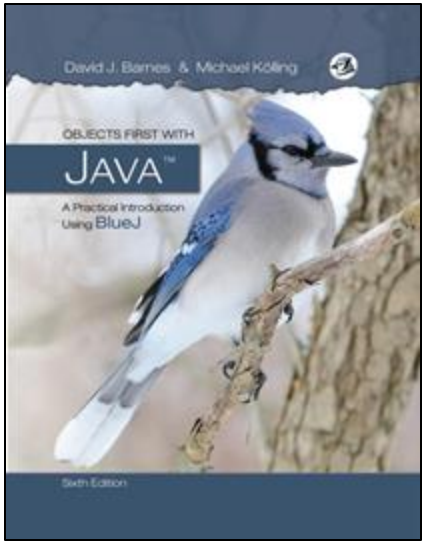
Unidad 3

Buen comportamiento de los objetos



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson del capítulo 9 del libro “Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition” escrito por David J. Barnes y Michael Kolling” © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Prevención y detección de errores
- Depuración (*debugging*)
- Pruebas (*testing*)
  - Automatización de pruebas



© Scott Adams, Inc./Dist. by UFS, Inc.

Conceptos fundamentales

- Prevención y detección de errores
- Depuración (*debugging*)
- Pruebas (*testing*)
  - Automatización de pruebas



© Scott Adams, Inc./Dist. by UFS, Inc.

Ejemplo

```
public void test()
{
    int sum = 1;

    for (int i = 0; i <= 4; i++);
    {
        sum = sum + 1;
    }

    System.out.println
    ("The result is: " + sum);
    System.out.println
    ("Double result: " + sum+sum);
}
```

¿Cuál es la salida?

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resultados posibles

¿Qué resultado se muestra?

```
The result is: 5
The result is: 6
The result is: 11
The result is: 2

Double result: 12
Double result: 4
Double result: 22
Double result: 66
```

¿Qué resultado se muestra?

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resultados posibles

¿Qué resultado se muestra?

```
The result is: 5
The result is: 6
The result is: 11
The
Doub
Doub
Double result: 22
Double result: 66
```

¿Qué resultado se muestra?

```
The result is: 2
Double result: 22
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo

```
public void test()
{
    int sum = 1;

    for (int i = 0; i <= 4; i++);
    {
        sum = sum + 1;
    }

    System.out.println
    ("The result is: " + sum);
    System.out.println
    ("Double result: " + sum+sum);
}
```

¿Qué resultado se muestra?

El punto y coma marca el final de una ***sentencia vacía***, el cuerpo del bucle.

Código que sigue al bucle. Con el ";", no es ***el cuerpo del bucle***.

```
The result is:  2
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo

```
public void test()
{
    int sum = 1;

    for (int i = 0; i <= 4; i++);
    {
        sum = sum + 1;
    }

    System.out.println
    ("The result is: " + sum);
    System.out.println
    ("Double result: " + sum+sum);
}
```

¿Qué resultado se muestra?

Esta suma nunca se hace – el número se convierte en una cadena y se une a la cadena ***(dos veces)***.

```
The result is:  2
Double result is: 22
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tenemos que trabajar con errores

- Los primeros errores suelen ser **errores sintácticos**.
  - Son detectados por el compilador.
- Los errores posteriores suelen ser **errores lógicos**.
  - El compilador no los puede detectar.
  - Se suelen conocer como "*bugs*" (*).
- Algunos errores lógicos *no aparecen inmediatamente o de una forma obvia* o *aparecen intermitentemente* sin un motivo obvio:
  - también conocidos como %&#&% *bugs* 
  - el software comercial rara vez está libre de errores.
    - a veces, es incluso mucho peor…

(*) https://es.wikipedia.org/wiki/Error_de_software

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Prevención vs Detección (Desarrollar vs Mantener)

- Podemos prevenir la aparición de errores:
  - encapsulación
  - cohesión* y acoplamiento*
- Podemos mejorar las posibilidades de detección:
  - modularización
  - buena documentación
- Podemos desarrollar habilidades de detección:
  - pruebas
  - depuración

* Nos centraremos en estos conceptos en la unidad 7

Prevención vs Detección (Desarrollar vs Mantener)

- Podemos **prevenir la aparición de errores**:
  - Usando técnicas de ingeniería del software:
    - *encapsulación* de datos en *campos* de un *objeto*.
    - prestando atención a cohesión* y acoplamiento*.
      - diseñar cada *clase* para que sea *responsable* de *una* sola cosa

* Nos centraremos en estos conceptos en la unidad 7

Prevención vs Detección (Desarrollar vs Mantener)

- Podemos **mejorar las posibilidades de detección**:
  - Usando prácticas de ingeniería del software:
    - modularizando correctamente el código
    - escribiendo la documentación *durante el Desarrollo*

```
/**
 * Create a machine that issues tickets of the given pric
 * Note that the price must be greater than zero, and the
 * are no checks to ensure this.
 */
public TicketMachine(int cost)
{
    price = cost;
    balance = 0;
    total = 0;
}
```

```
/**
 * TicketMachine mo els a naive ticket machine that issues
 * flat-fare tickets
 * The price of a ti ket is specified via the constructor.
 * It is a naive mach ne in the sense that it trusts its users
 * to insert enough mo ey before trying to print a ticket.
 * It also assumes that users enter sensible amounts.
 *
 * @author David J. Barn s and Michael Kölling
 * @version 2016.02.29
 */
public class TicketMachine
{
    // The price of a ticket from this machine.
    private int price;
    // The amount of money entered by a customer so far.
    private int balance;
```

Pearson

Copyright © 201

Prevención vs Detección (Desarrollar vs Mantener)

- Podemos **desarrollar habilidades de detección**:
  - Usando técnicas de ingeniería del software
    - planificando y realizando **pruebas** que **detecten** la **presencia** de errores *durante el Desarrollo*
      - ej. Interactuando con clases/objetos/métodos usando las utilidades de BlueJ
      - creando un conjunto de clases de prueba JUnit
    - **depurando** para **localizar** el **origen** de los errores *durante el Desarrollo*
      - La manifestación de un error puede ocurrir a cierta 'distancia' de su origen.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Prevención y detección de errores
- Depuración (*debugging*)
- Pruebas (*testing*)
  - Automatización de pruebas



© Scott Adams, Inc./Dist. by UFS, Inc.

Técnicas de depuración

- OK, encontramos errores, ¿cómo los resolvemos?
  - ***recorridos manuales***:
    - *¡lee tu código!*
    - *¿por qué debería funcionar? [**No:** ¡¡¿por qué no funciona?!!]*
  - ***recorridos verbales***:
    - *ayuda explicarlo a otra persona …*
  - si no vemos la luz, *¡¡mira qué ocurre!!!* :
    - los programadores inexpertos *añaden **sentencias print** a su código para mostrar el estado de las variables (en puntos clave) y comprueban si muestran lo que esperaban*
    - los programadores expertos *ejecutan su código con un **depurador** interactivo …*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Recorridos manuales

- Revisión de copias impresas/digitales del Código Fuente como si fuéramos a leer un libro
- Relativamente infrautilizado.
  - Un enfoque de baja tecnología.
  - Más potente que apreciado.
- ¡Aléjate del ordenador!
- 'Ejecuta' un programa a mano.
- Vistas de alto nivel (*Step*) o bajo nivel (*Step into*).

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Recorridos verbales

- Explicar a alguien qué hace el código:
  - **Ell@s** podrían detectar el error.
  - **Tu** puedes detectar el error, mediante la explicación.
    - *¡explica por qué debería funcionar!*
      - ***[no preguntes**: ¡¿por qué no funciona?!!]*



☺ Rubber Duck Debugging ☺



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Depuradores

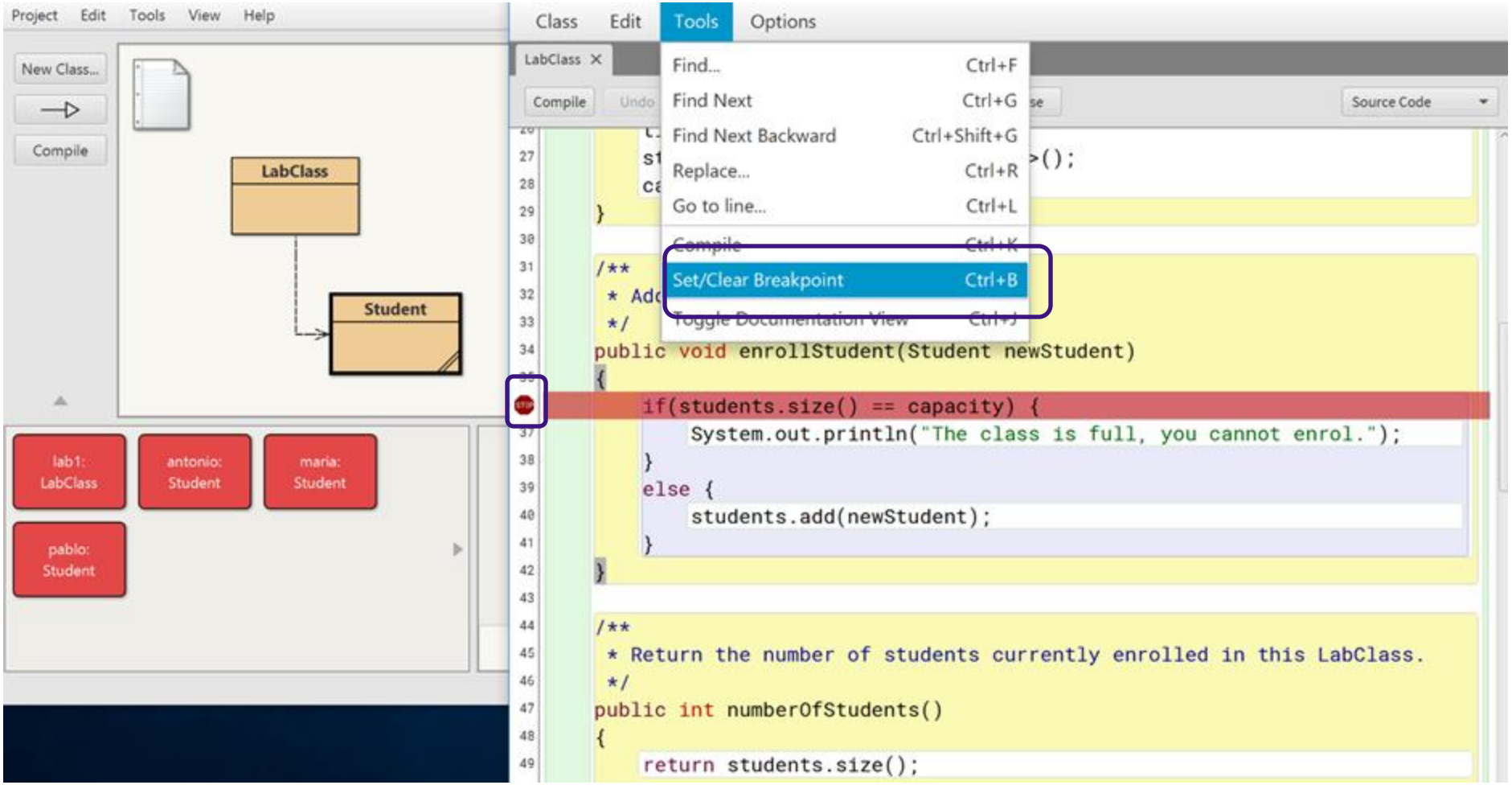
- Útil para comprender el comportamiento de un programa…
- … tanto si hay o no un error de programa.
- Los depuradores son específicos del entorno y del lenguaje.
  - BlueJ tiene un depurador integrado.
    - Soporta *breakpoints* (puntos de interrupción).
    - ***Step*** y ***Step-into*** permiten una ejecución controlada.
    - Secuencia de llamadas (*stack*).
    - Muestra el estado del objeto.
    - Pasos básicos:
      - Fijar puntos de ruptura.
      - Recorrer el código.
      - Examinar variables.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Depurador en BlueJ

- Pasos básicos:
- Fijar puntos de ruptura.

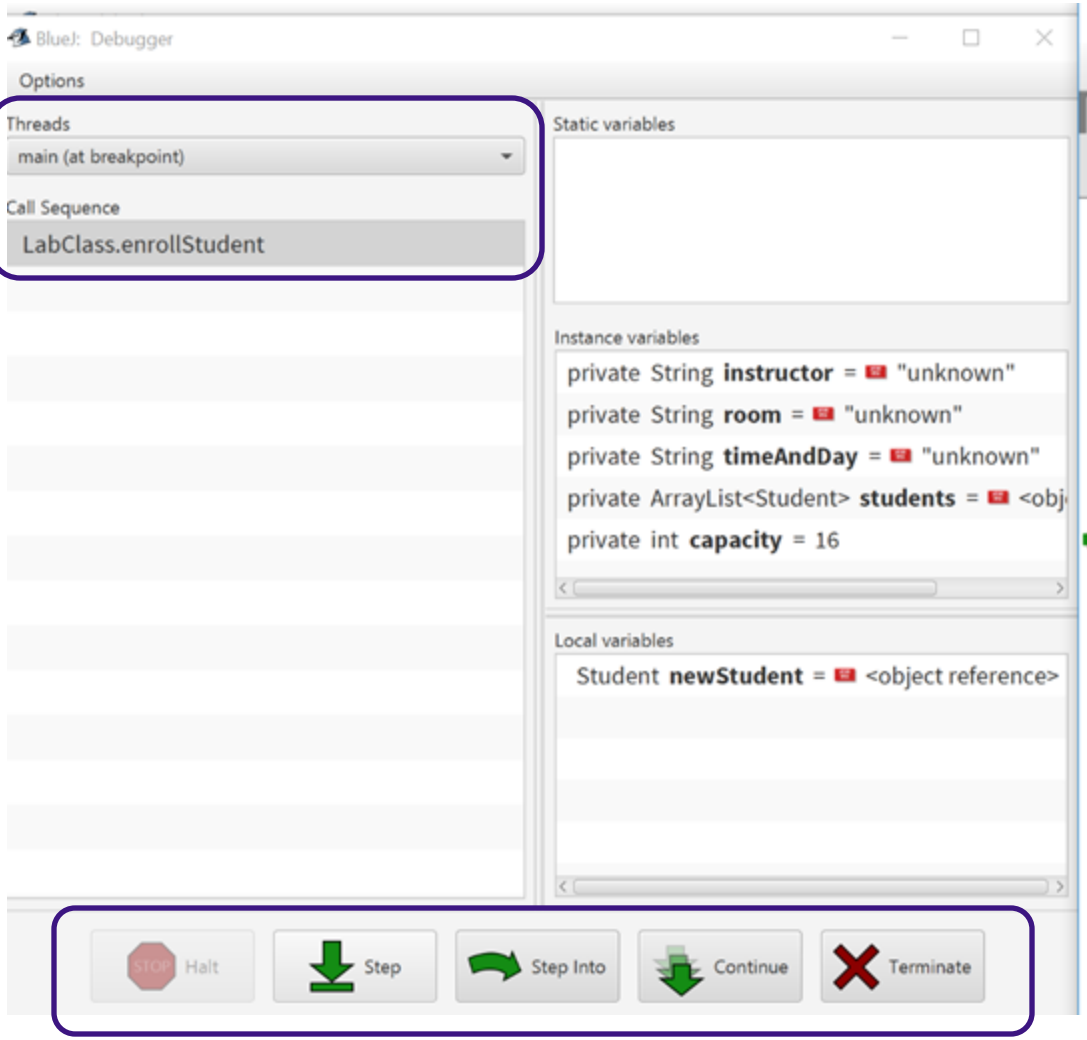


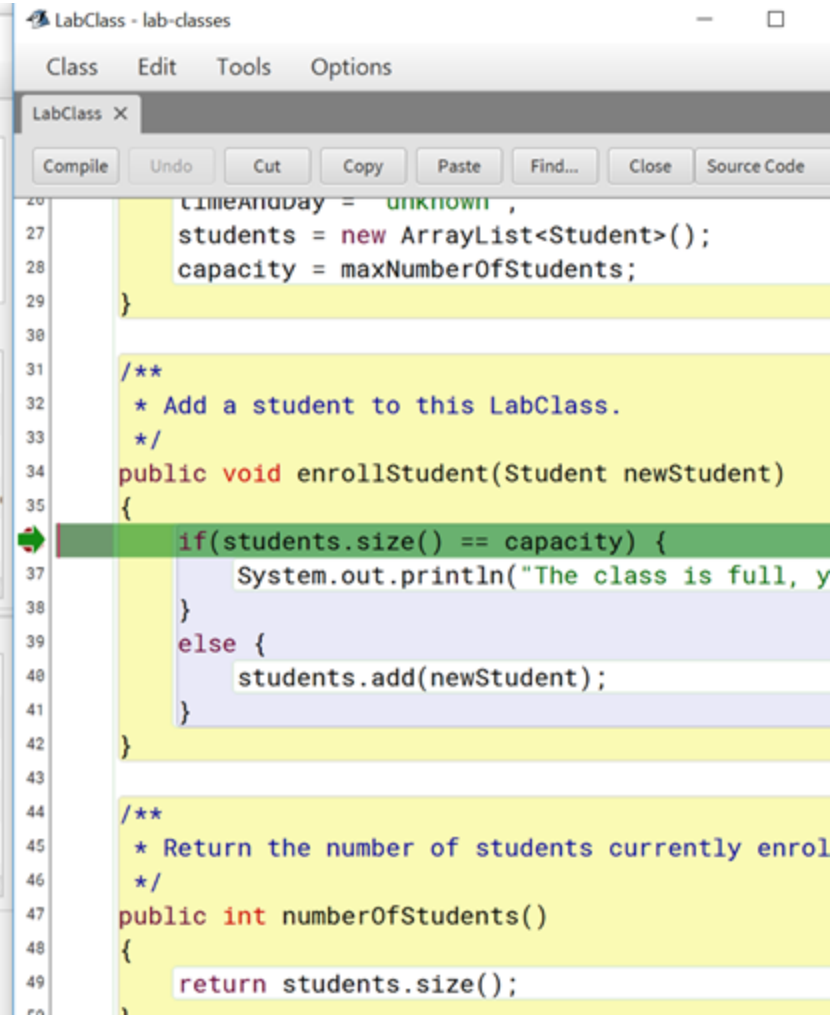
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Depurador en BlueJ

- Pasos básicos:
- Recorrer el código.



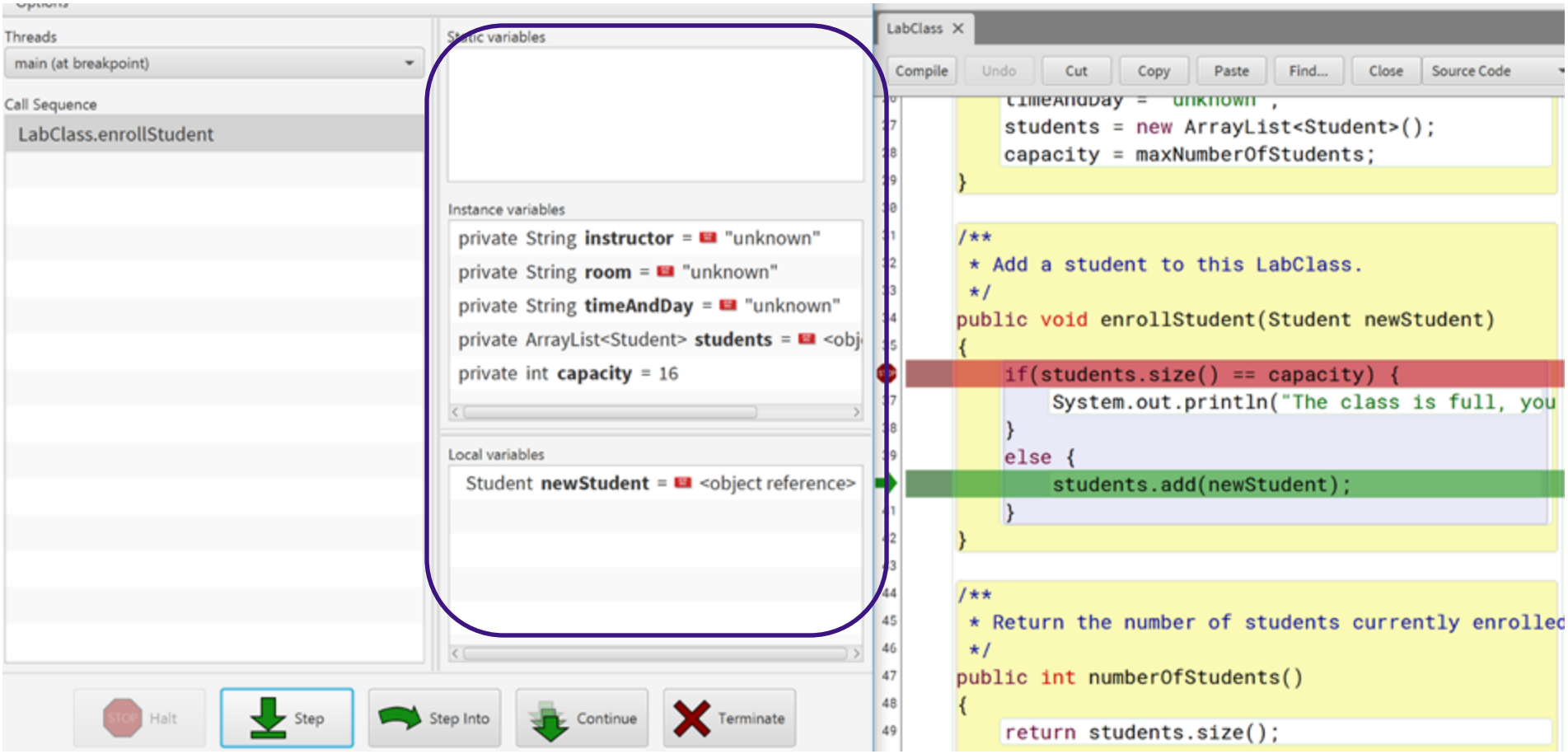


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Depurador en BlueJ

- Pasos básicos:
- Examinar variables.



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

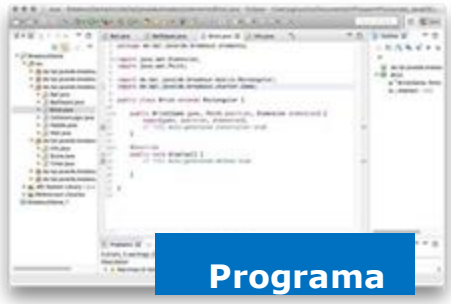
- Prevención y detección de errores
- Depuración (*debugging*)
- Pruebas (*testing*)
  - Automatización de pruebas



© Scott Adams, Inc./Dist. by UFS, Inc.

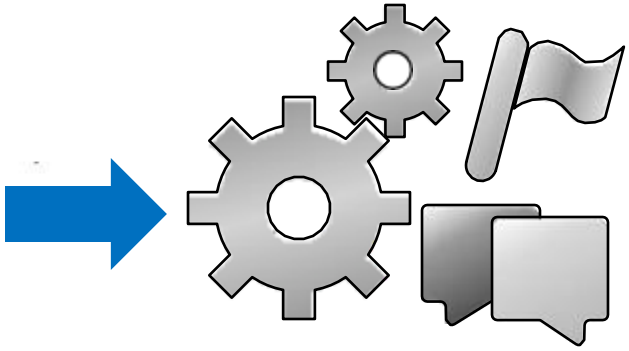
Pruebas vs. ensayo y error





- Cantidad de código manejable
- Todas las funcionalidades comprobadas por ensayo y error
- Se va entregando el código





- Se solicita una nueva característica
- Una característica más
- Y otra característica más
- Refactorización
- ¿Comprobar todas las funcionalidades una, y otra vez y otra vez más…?
- ¿¿Qué puede ir mal??

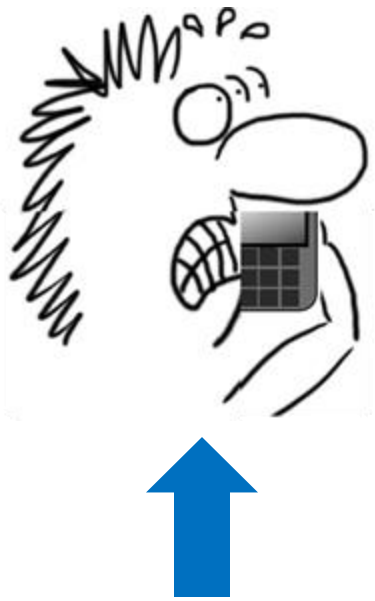
Pearson

Copyright © 2017, 2012, 200

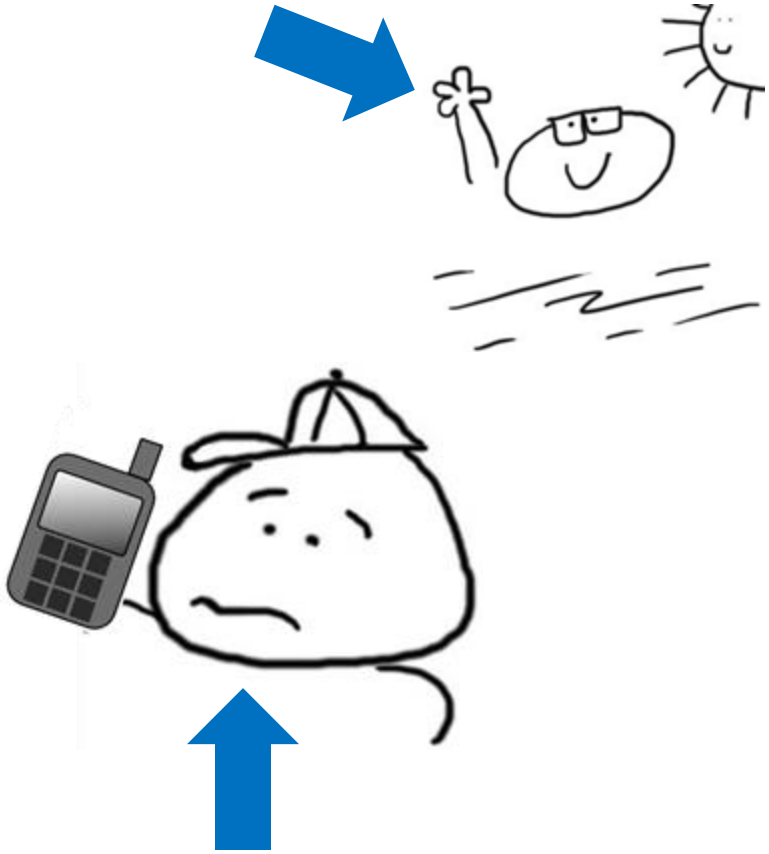
Pruebas vs. ensayo y error

Dr. Dem Anding yéndose de vacaciones tan feliz porque las pruebas eran responsabilidad de Dev Eloper

¿¿Qué podía ir mal??



Jeff Az@, Director/a general de la empresa al que se le acaba de colgar la demo del programa en la rueda de prensa de presentación



Dev Eloper temiendo, al ver quién llama, que le va a tocar trabajar varios días sin dormir mientras se arrepiente de no haberse hecho influencer para vivir del cuento

Pruebas estáticas vs. dinámicas

Pruebas estáticas

- Análisis de código fuente (herramientas)
- Revisiones de código (compañer@s)







Pruebas dinámicas

- El código es ejecutado
- Los resultados reales se comparan con los esperados



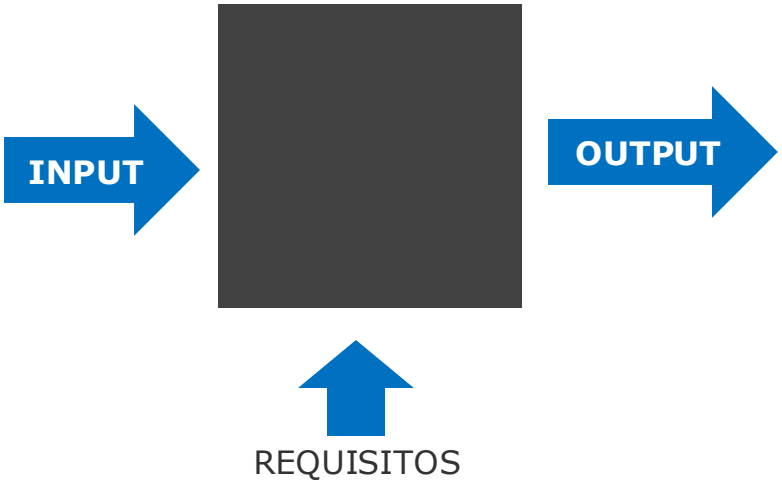


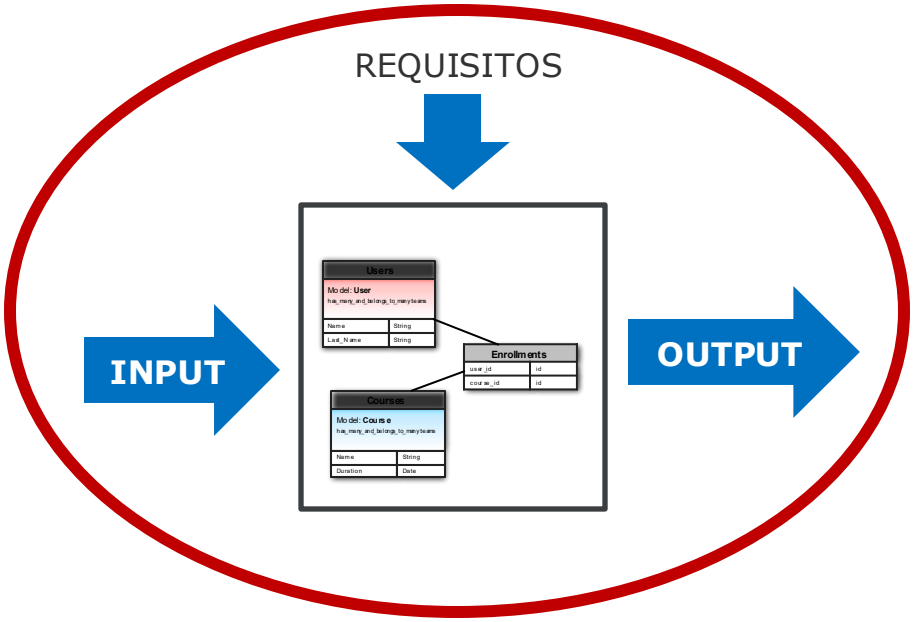
TestNG

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pruebas de caja negra vs. pruebas de caja blanca





Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Niveles de pruebas (o tests)

Pruebas unitarias

Tests de integración

Tests del sistema

Tests de aceptación



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipos de pruebas (o tests)

- Pruebas de regresión
  - Comprueba si el código cumple aún los requisitos después de "refactorizar" (Pruebas de unidad/integración)
- Pruebas de comportamiento
  - Consultar al cliente si se cumple el comportamiento esperado
- Pruebas de rendimiento
  - ¿Es la aplicación lo suficientemente rápida? (gran cantidad de datos)
- Pruebas continuas
  - CI –server (servidor de Continous Integration)
- Pruebas de facilidad de uso
  - Eye-tracking, mouse-tracking







Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pruebas unitarias

- *Pruebas de aplicación* pueden realizarse solo cuando la aplicación *completa* ha finalizado

- *Pruebas de unidad* pueden realizarse para una única unidad una vez haya sido escrita y compilada
  - Se puede probar cada unidad de una aplicación:
  - *cada método*
  - *cada clase*
  - *cada paquete (package)*

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pruebas unitarias

- *Pruebas de unidad* pueden realizarse para una única unidad una vez haya sido escrita y compilada
  - Se puede probar cada unidad de una aplicación.
  - Puede (debe) realizarse durante el desarrollo:
    - encontrar y corregir errores pronto disminuye los costes de desarrollo (ej. tiempo de programación).
    - encontrar problemas después del despliegue es muy caro.
      - es lo que ocurre normalmente.
    - si el diseño del sistema carece de cohesión y tiene un alto acoplamiento entre unidades, puede que no encontremos nunca los *bugs* o, si se encuentran, no puedan corregirse.
    - **debe hacerse un conjunto de pruebas.**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Fundamentos de las pruebas

- Entender qué debería hacer la unidad – su **contrato** (requisitos).
  - buscar el cumplimiento *(pruebas positivas)*
  - buscar inclumplimiento *(pruebas negativas)*
    - incluyendo null y condiciones vacías
- **Límites** en pruebas.
  - Cero, uno, completo.
    - Buscar en una colección vacía.
    - Añadir un elemento a una colección llena.
    - Buscar/eliminar el único elemento.

¡ Es un error muy común en personas inexpertas en pruebas realizar solo pruebas *positivas* !

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Prevención y detección de errores
- Depuración (*debugging*)
- Pruebas (*testing*)
  - Automatización de pruebas



© Scott Adams, Inc./Dist. by UFS, Inc.

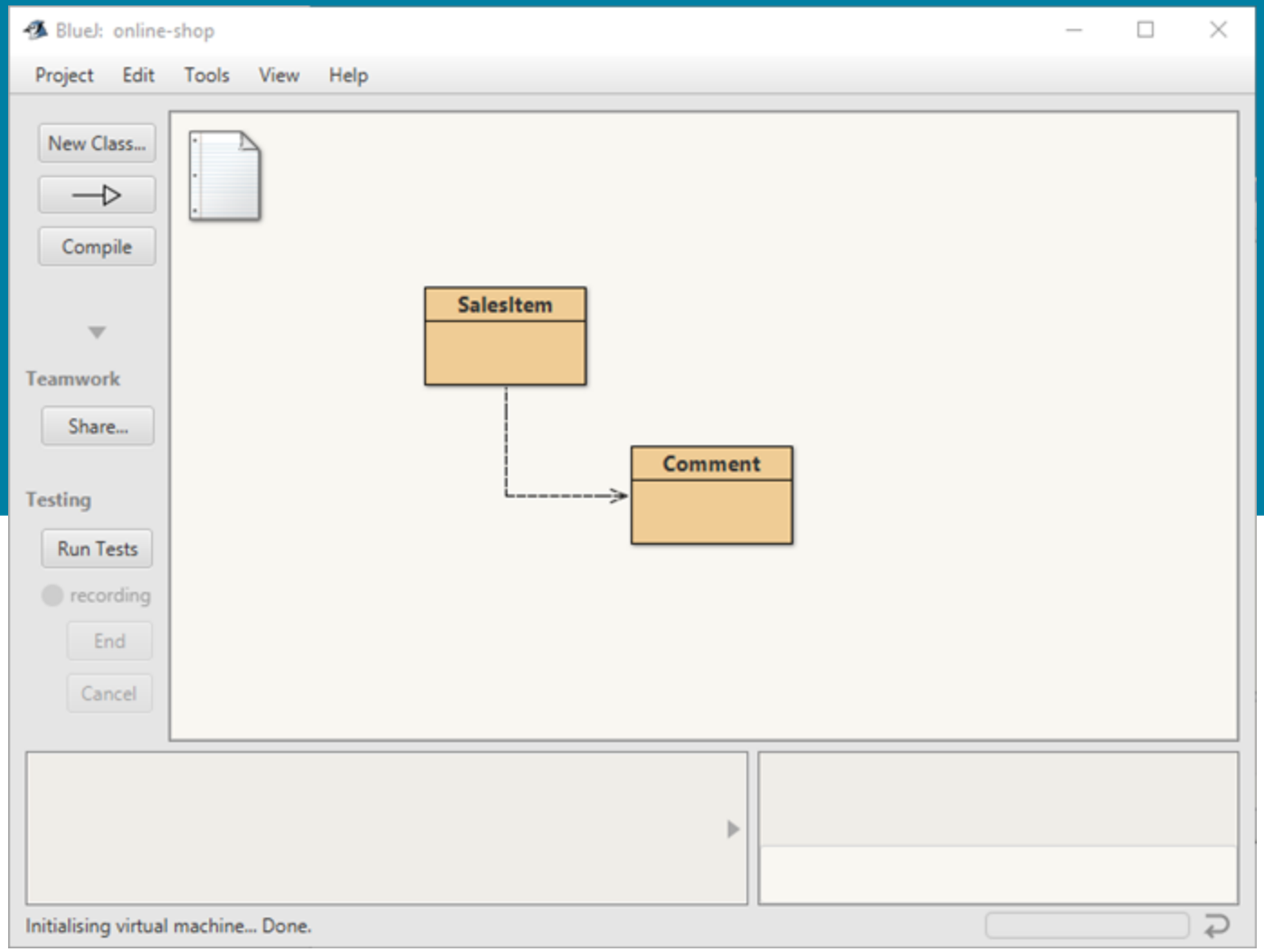
Pruebas unitarias en BlueJ

- Se pueden crear objetos de clases individuales.
- Se pueden invocar métodos individuales.
- Los inspectores de objetos proporcionan una vista actualizada del estado de un objecto.
- Exploremos el proyecto **online-shop-junit**.
  - *¡Contiene errores!*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo del proyecto Online-Shop-junit



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo del proyecto Online-Shop-junit

- Se pueden crear objetos de tipo SalesItem con name y price.
- Objetos de tipo Comment incluyen text, author (nombre del autor), votes y una clasificación (rating).
  - rating tiene un rango de 1 a 5 (inclusive).
- Cada persona puede dejar solo un comentario.
  - Los intentos sucesivos de dejar más comentarios por el mismo autor son rechazados.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo del proyecto Online-Shop-junit

- La interface de usuario (aún no implementada en este proyecto) incluirá la pregunta "¿ha sido útil este comentario?" con dos botones Yes y No.
  - Un click de usuario Yes o No es votar a favor (upvoting) o en contra (downvoting) del objeto Comment.
  - El balance de votos positivos (up) y negativos (down) se mantiene para los comentarios, de modo que los comentarios más útiles (los de mayor balance de votes) se pueden mostrar en la parte superior.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo del proyecto Online-Shop-junit

Para nuestras pruebas, nos concentraremos en la clase SalesItem.

- Los objetos de esta clase representan un solo artículo, incluyendo todos los comentarios hechos para ese artículo.
- Como parte de nuestra prueba, deberíamos comprobar algunas partes de su functionalidad, incluyendo:
  - ¿Se pueden añadir y eliminar comentarios para un artículo?
  - ¿Muestra correctamente el método *showInfo* toda la información almacenada sobre un artículo?
  - ¿Se cumplen correctamente las restricciones (constraints) (ratings entre 1 y 5, solo un comentario por persona)?
  - ¿Podemos encontrar correctamente el comentario más útil (el que tiene más votos)?

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Automatización de pruebas

- Usar un **juego de pruebas** pueden aliviar la carga de trabajo.
  - Se suelen escribir clases de prueba adicionales para automatizar las pruebas (ej.: un grupo de personas crean clases demo de prueba)
- Existen marcos de prueba para ayudar en la automatización.
- Veamos una automatización en el proyecto **online-shop-junit**.
  - Sólo hay que intervenir cuando se informa de un fallo (*failure*).

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

JUnit

- **JUnit** es un Java test framework (incluido en BlueJ)
- **Los casos de prueba** son métodos que contienen pruebas
- **Las clases de prueba** contienen métodos de prueba
- **Las aserciones (assertions)** se usan para confirmar los resultados esperados por los métodos

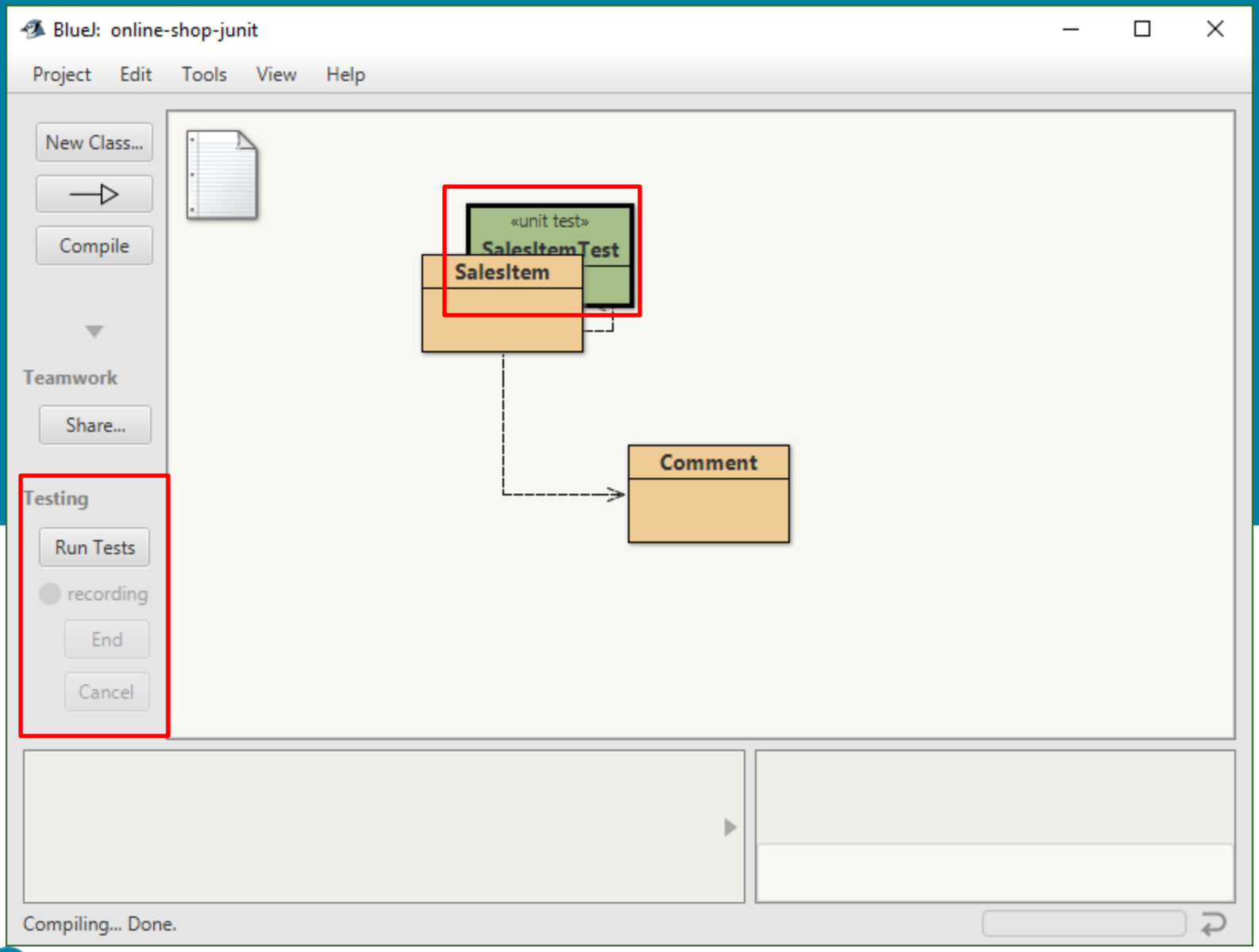
https://junit.org/junit5/docs/5.0.1/api/org/junit/jupiter/api/Assertions.html

- **Fixtures** se usan para realizar múltiples pruebas

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

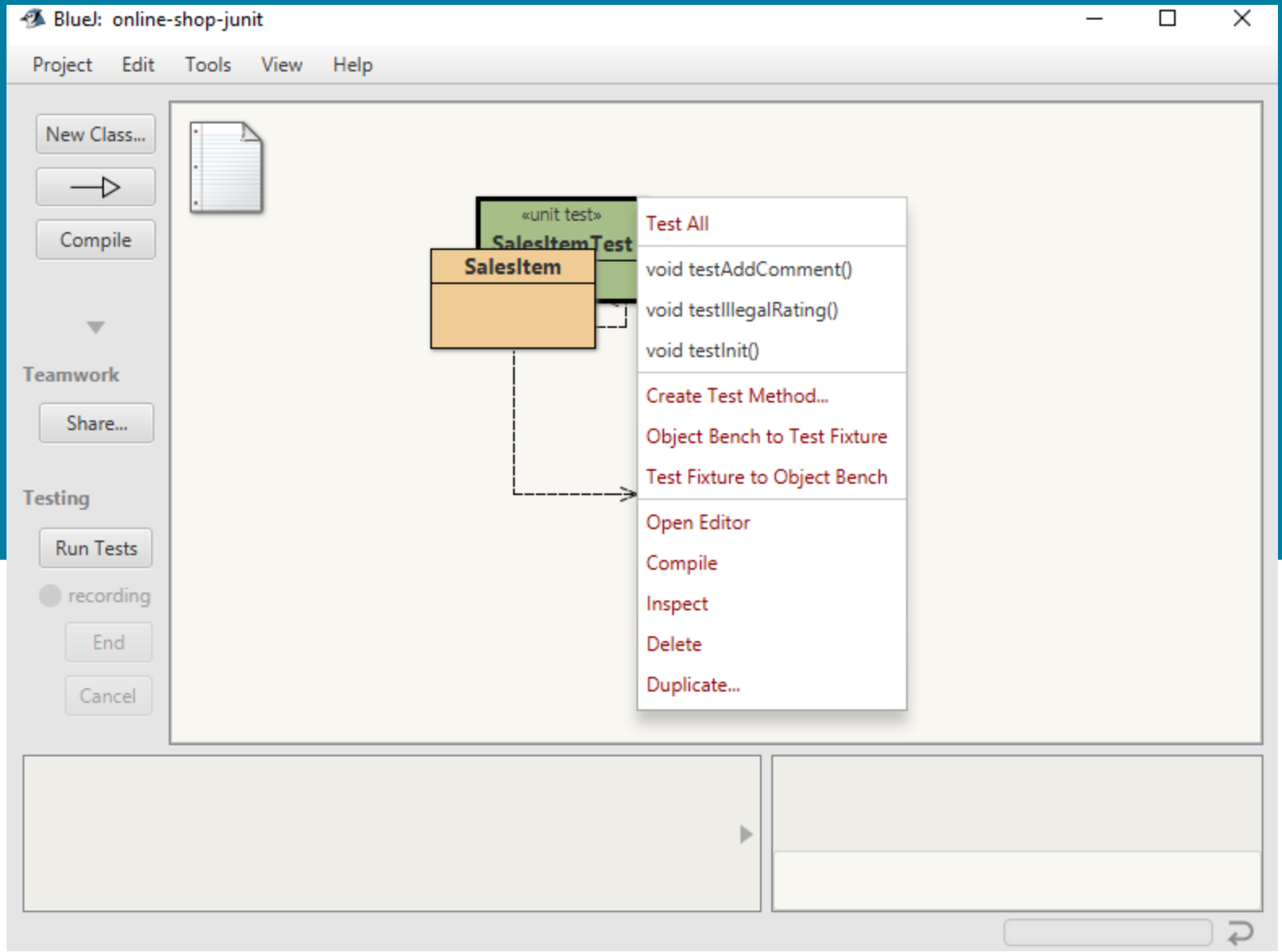
Demo de Online-Shop-Junit



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

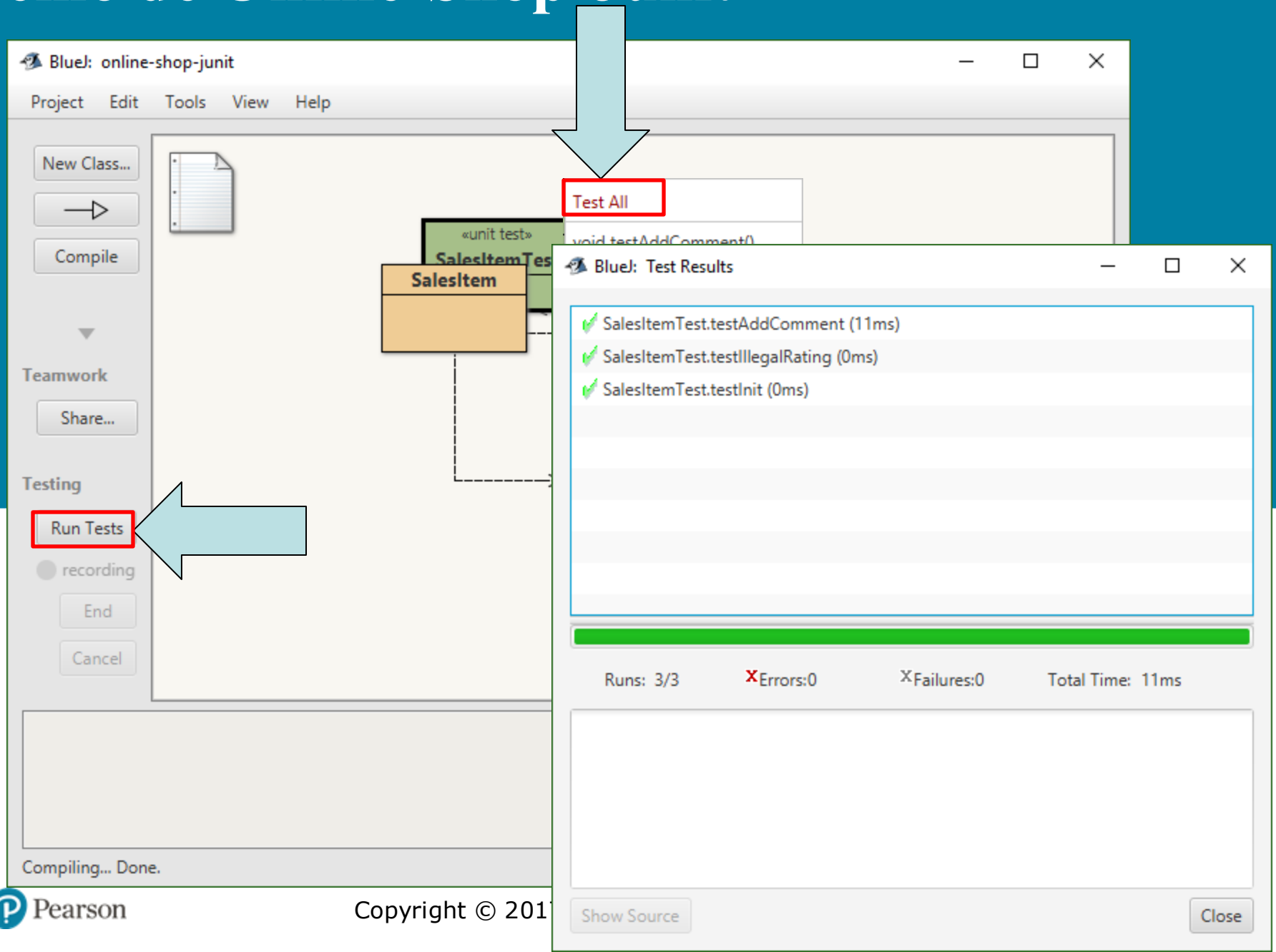
Demo de Online-Shop-Junit



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit



Pearson Copyright © 201

Demo de Online-Shop-Junit

SalesItemTest

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
1  import static org.junit.Assert.*;
2  import org.junit.After;
3  import org.junit.Before;
4  import org.junit.Test;
5
6  /**
7   * The test class SalesItemTest.
8   *
9   * @author  mik
10  * @version 0.1
11  */
12 public class SalesItemTest
13 {
14     /**
15      * Default constructor for test class SalesItemTest
16      */
17     public SalesItemTest()
18     {
19     }
20
21     /**
22      * Sets up the test fixture.
23      *
24      * Called before every test case method.
25      */
26     @Before
27     public void setUp()
28     {
29     }
```

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit

SalesItemTest

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
41      /**
42       * Test that a comment can be added, and that the comment count is correct afterwards.
43       */
44      @Test
45      public void testAddComment()
46      {
47          SalesItem salesIte1 = new SalesItem("Java for complete Idiots", 21998);
48          assertEquals(true, salesIte1.addComment("James Duckling", "This book is great. I lear
49          assertEquals(1, salesIte1.getNumberOfComments());
50      }
51
52      /**
53       * Test that a comment using an illegal rating value is rejected.
54       */
55      @Test
56      public void testIllegalRating()
57      {
58          SalesItem salesIte1 = new SalesItem("Java For Complete Idiots, Vol 2", 19900);
59          assertEquals(false, salesIte1.addComment("Joshua Black", "Not worth the money. The fc
60      }
61
62      /**
63       * Test that a sales item is correctly initialised (name and price).
64       */
65      @Test
66      public void testInit()
67      {
68          SalesItem salesIte1 = new SalesItem("test name", 1000);
69          assertEquals("test name", salesIte1.getName());
70          assertEquals(1000, salesIte1.getPrice());
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

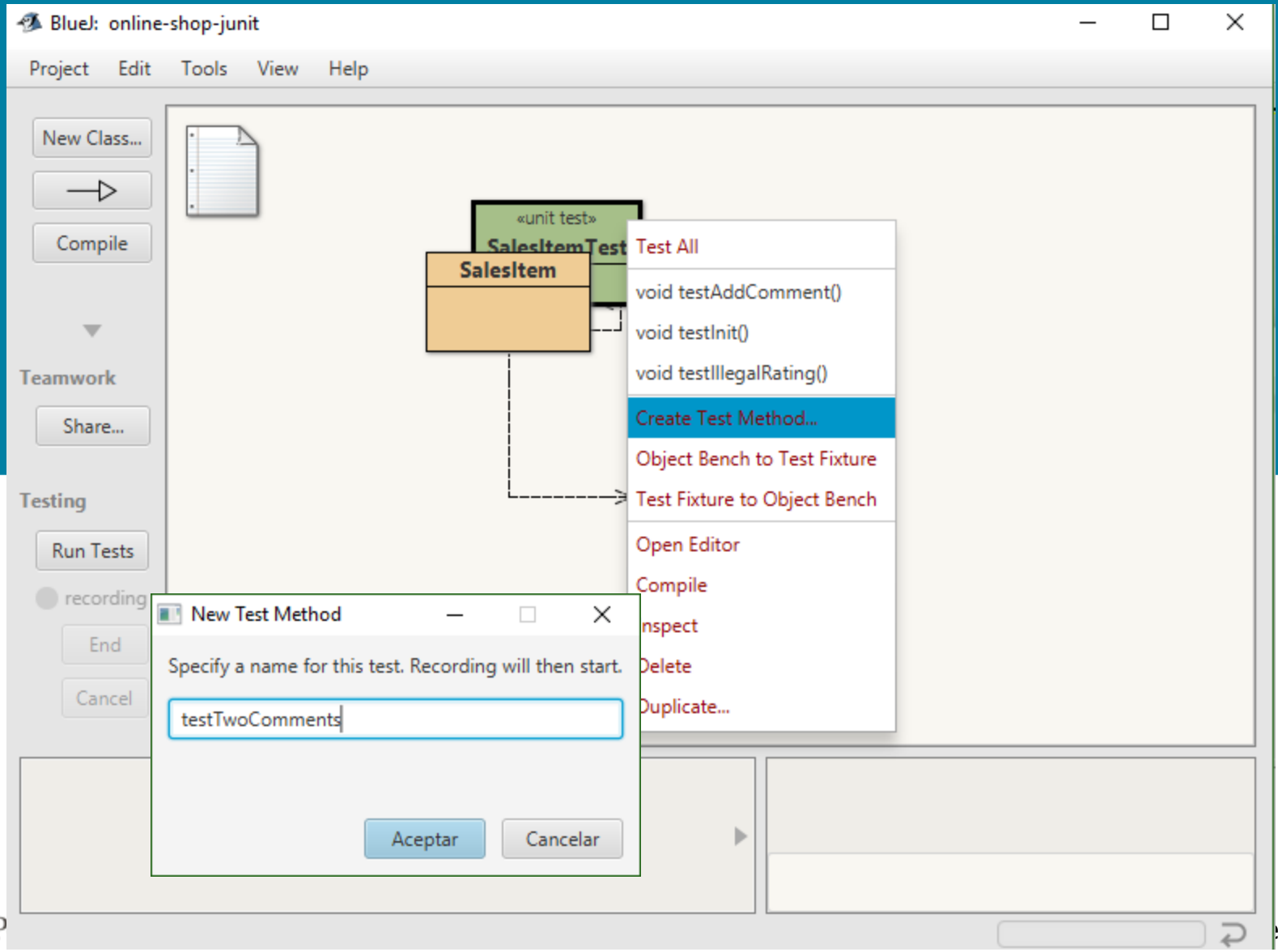
Grabando pruebas

- Es deseable la automatización de pruebas
  - crear y recrear pruebas manualmente requiere mucho tiempo
- BlueJ
  - hace posible combinar:
    - la efectividad de las pruebas unitarias de forma manual
    - el poder de la automatización de pruebas
  - permitiéndonos:
    - grabar pruebas de forma manual
    - reproducirlas más tarde para realizar pruebas de regresión

Pearson

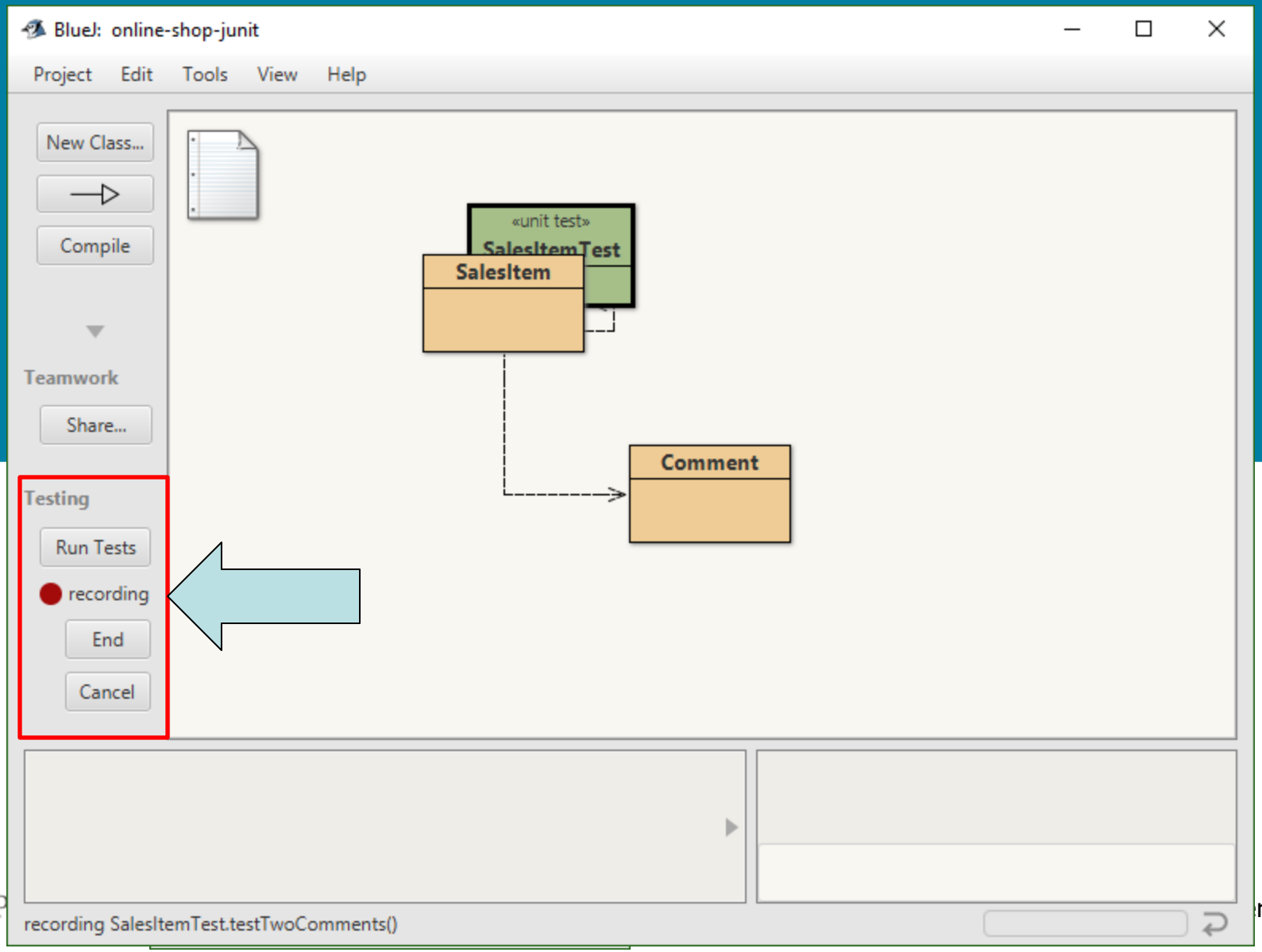
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



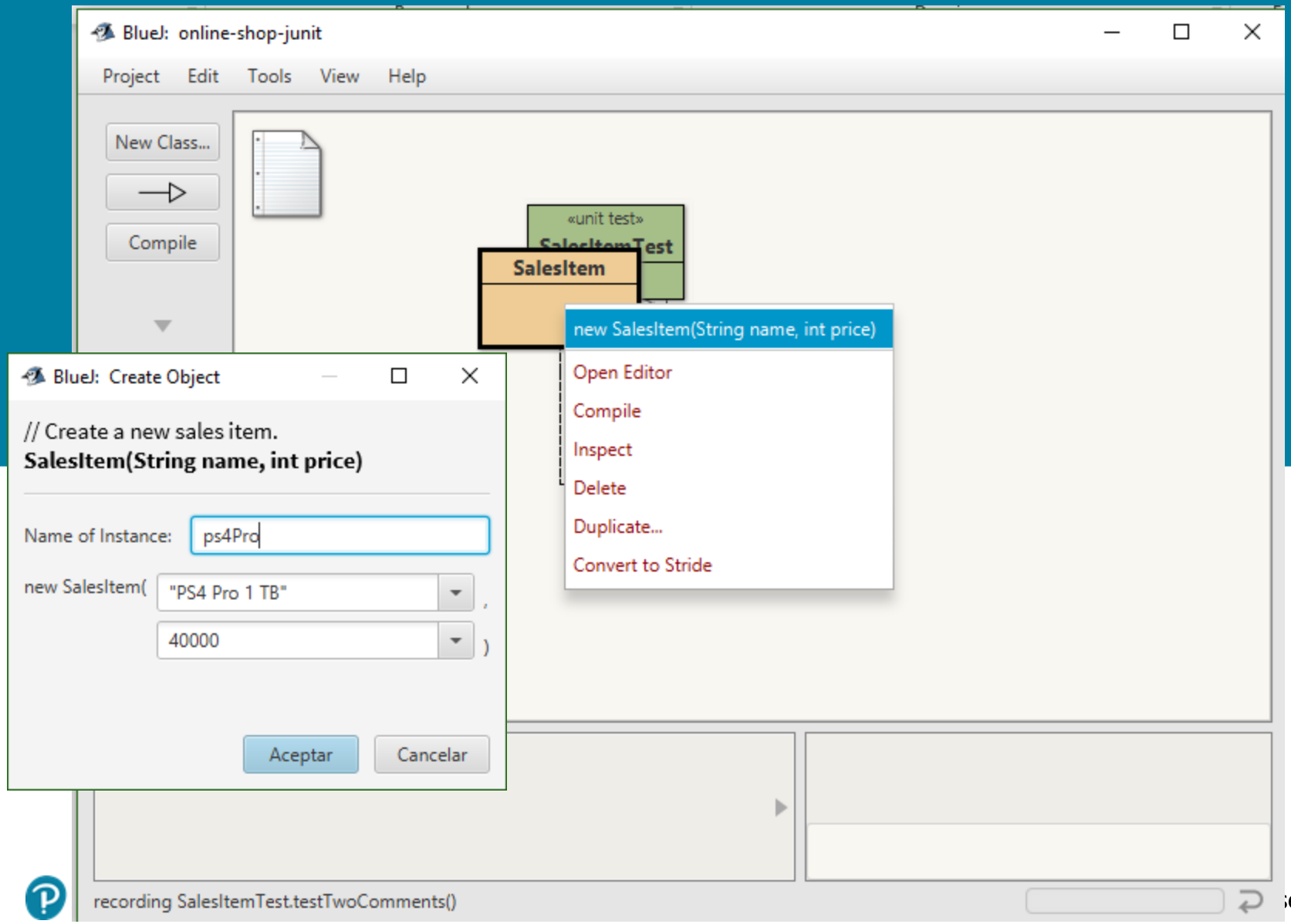
erved

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



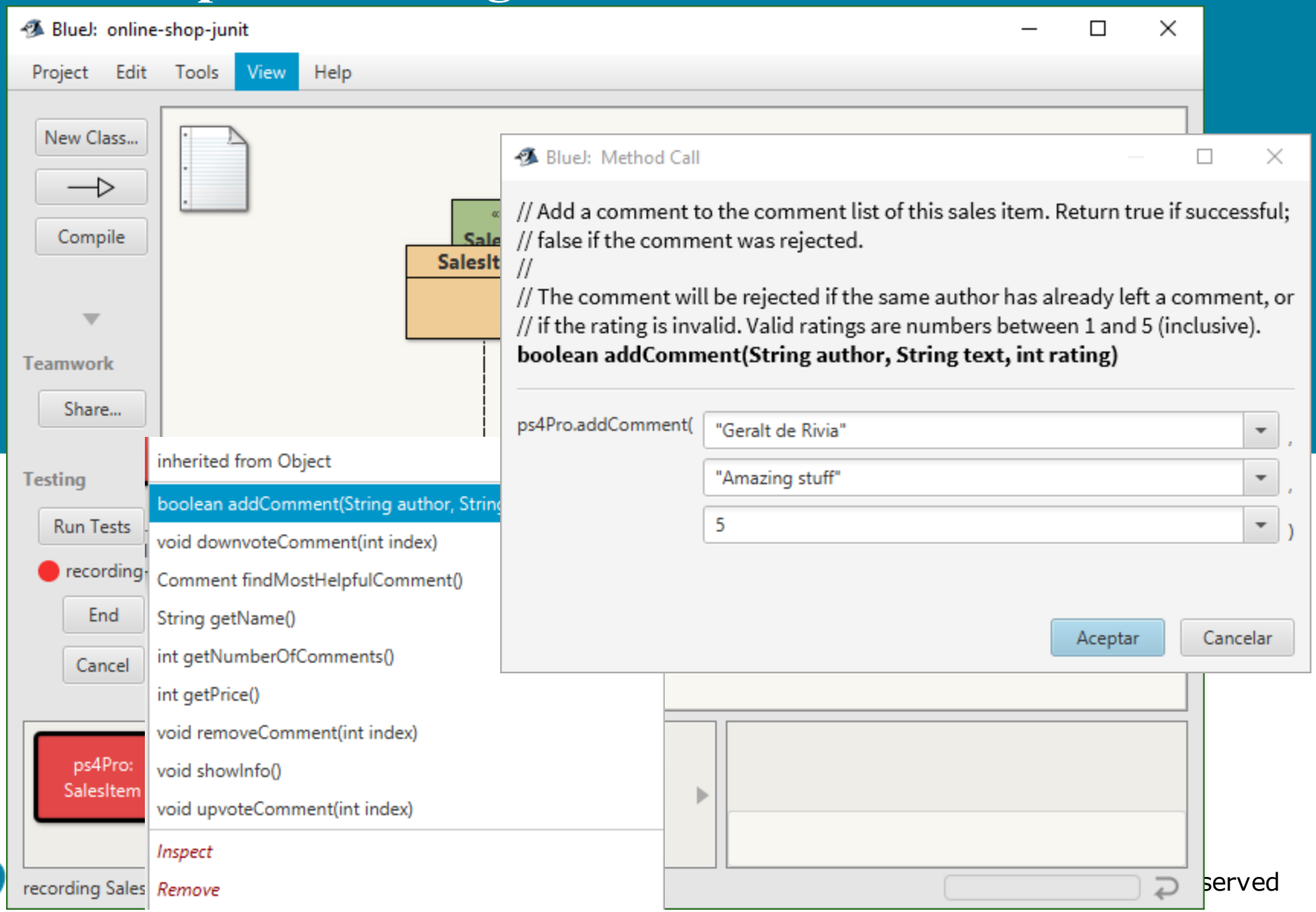
erved

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



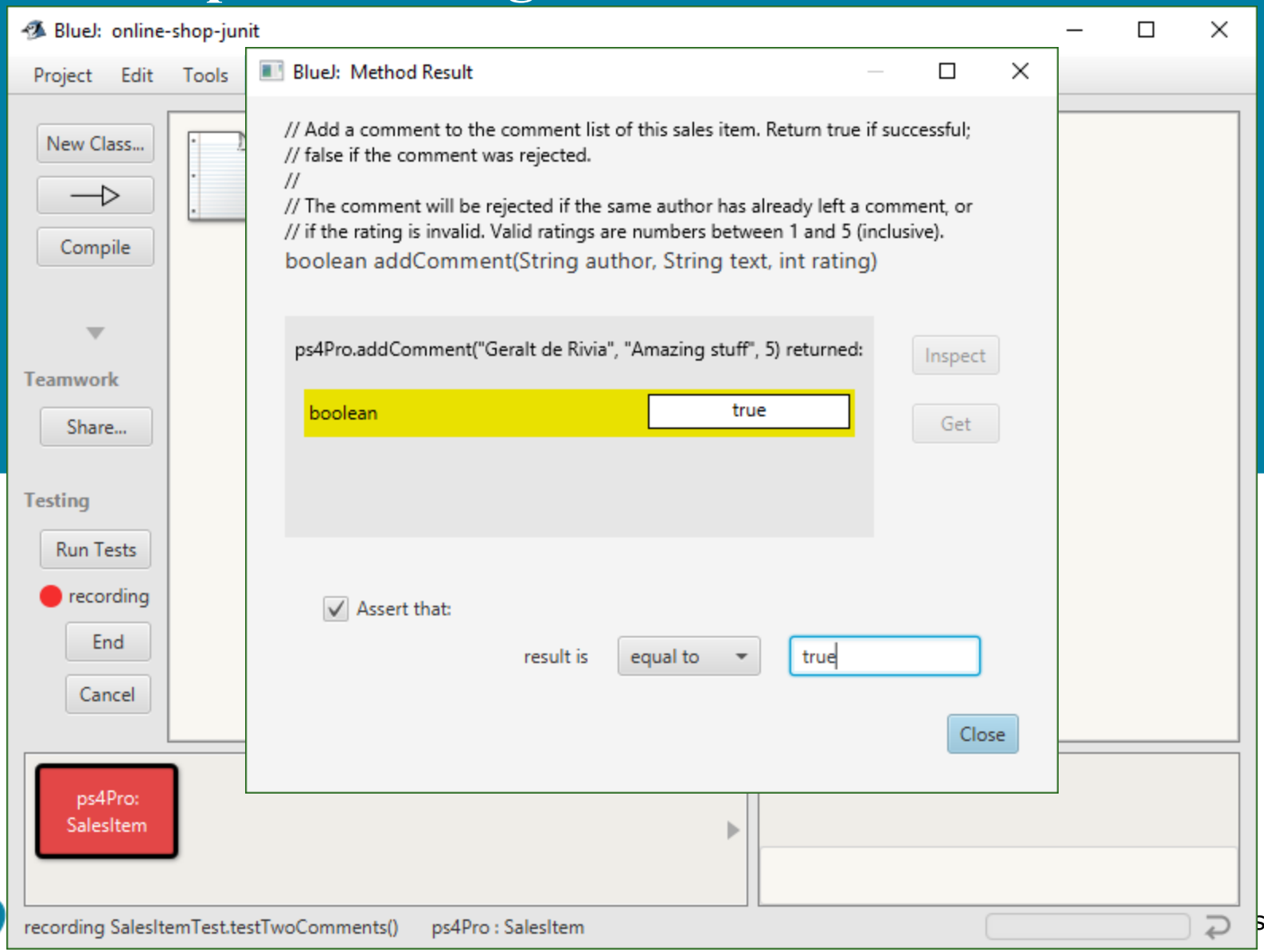
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



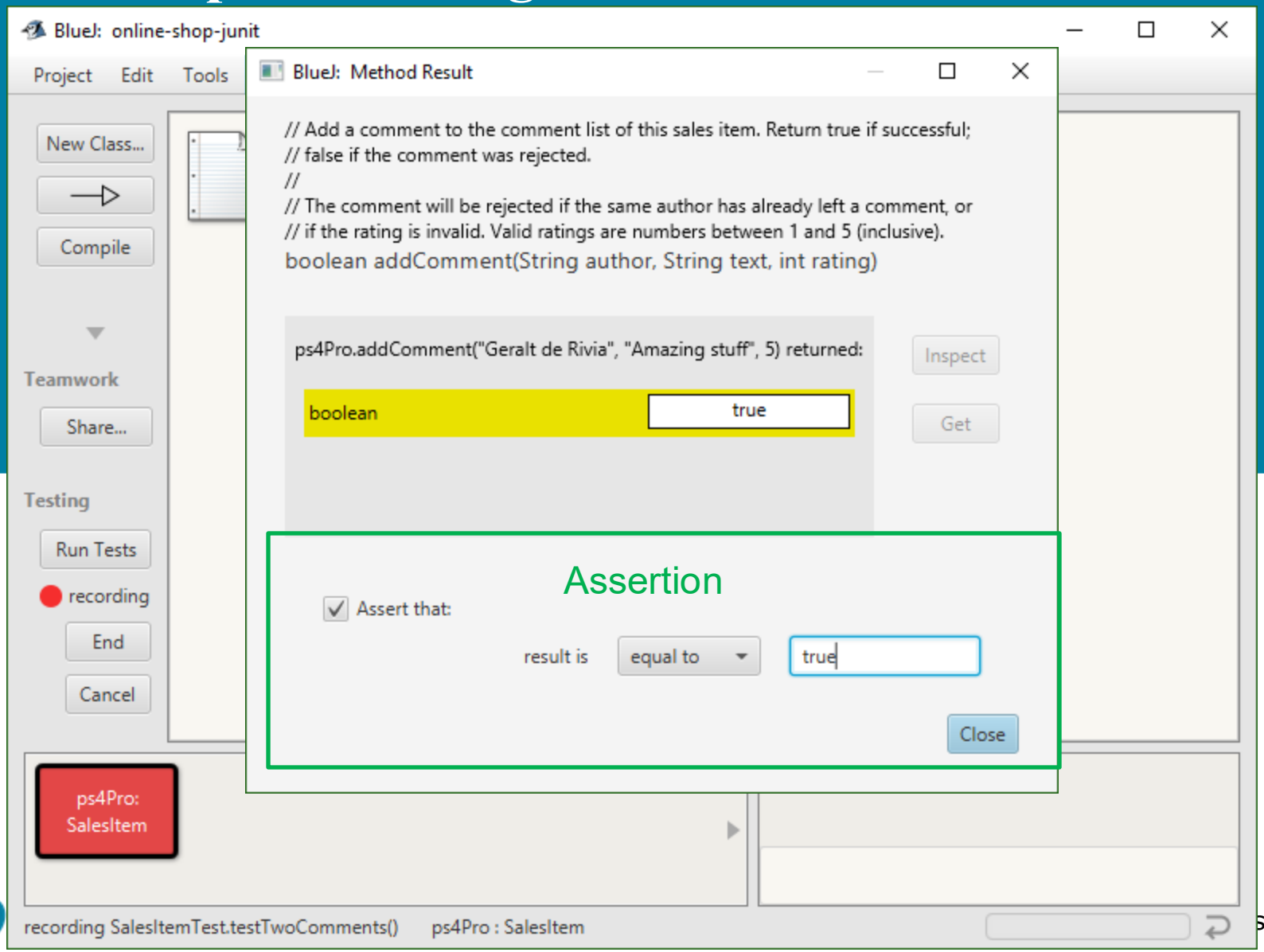
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



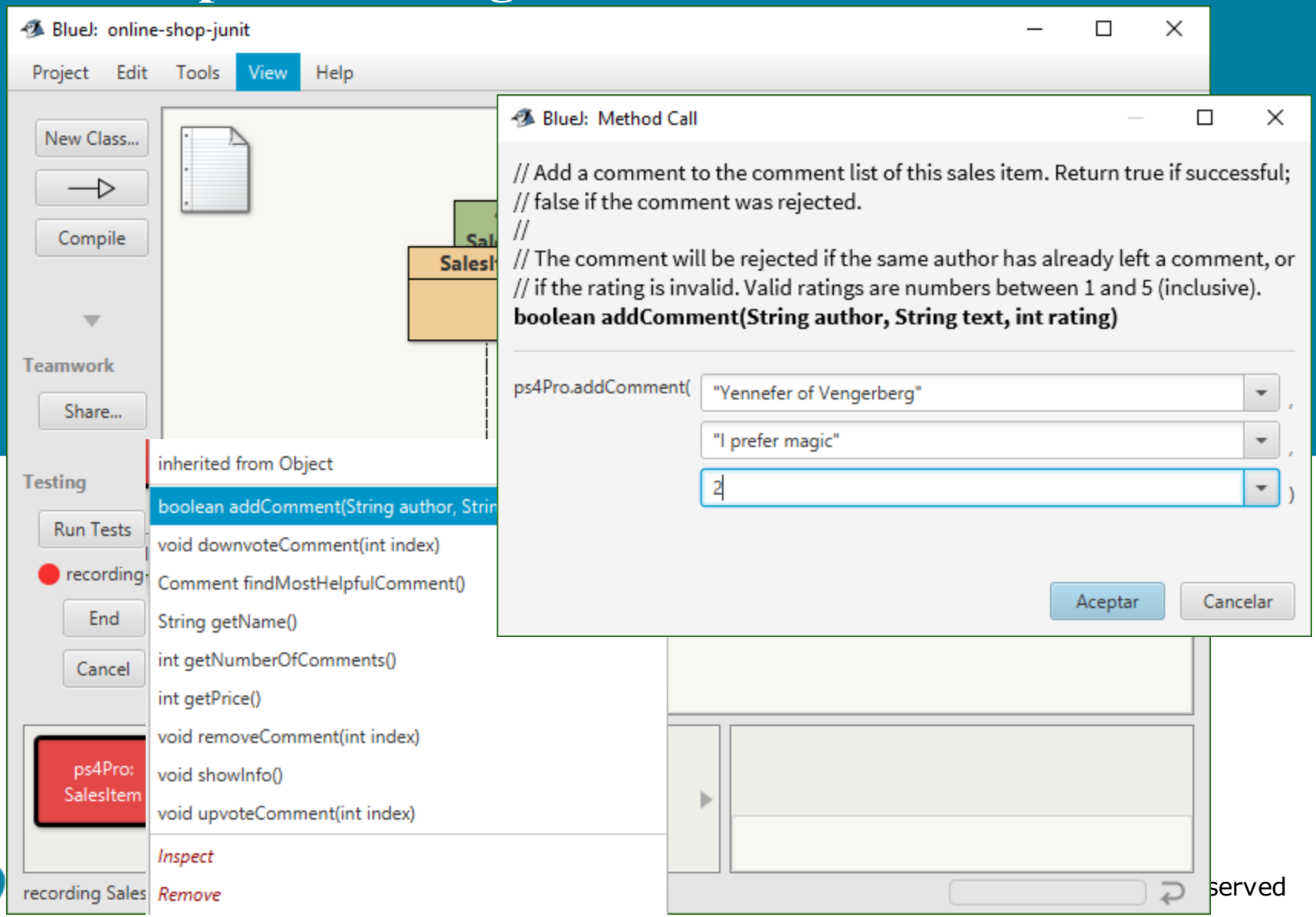
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



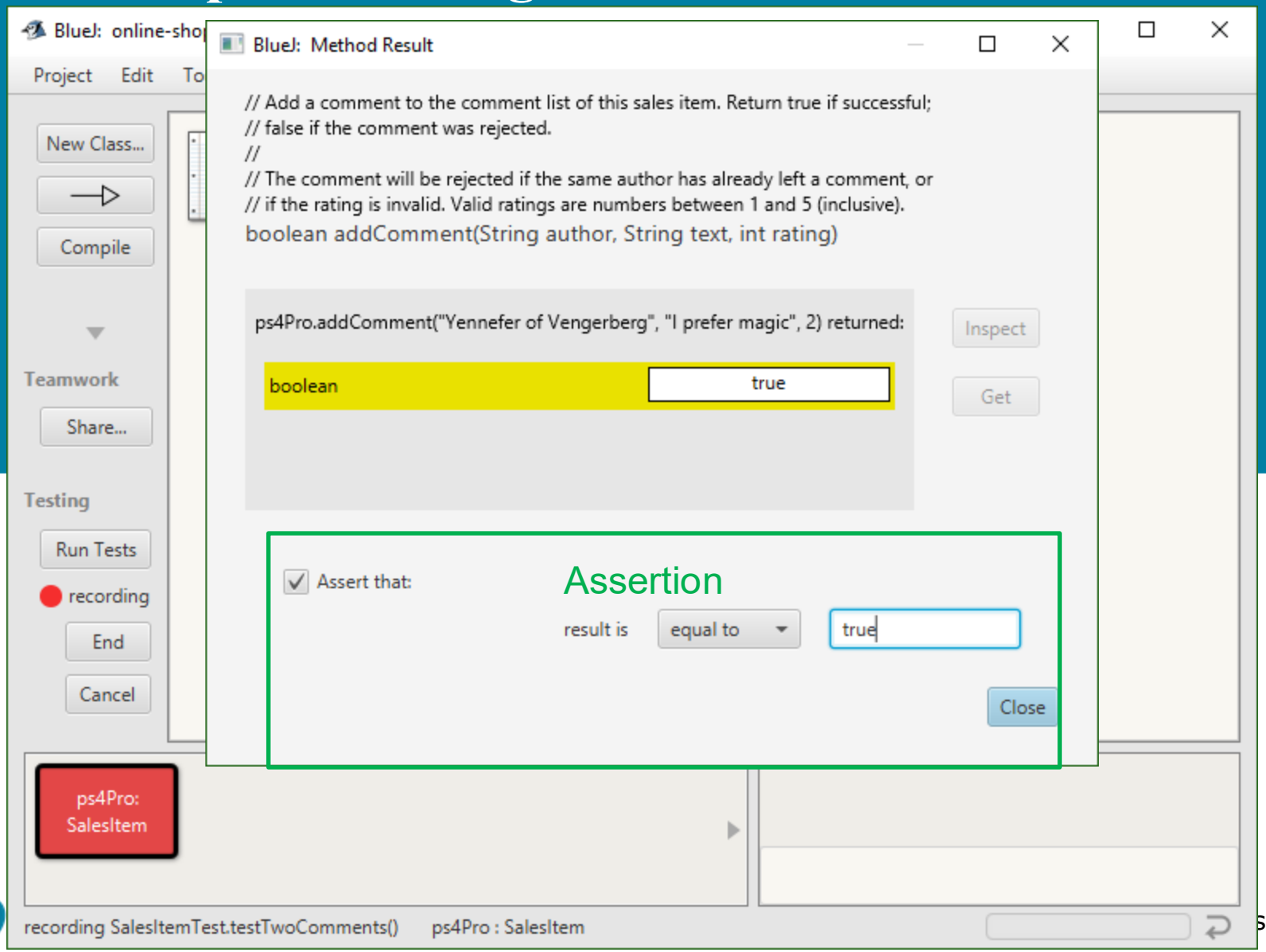
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



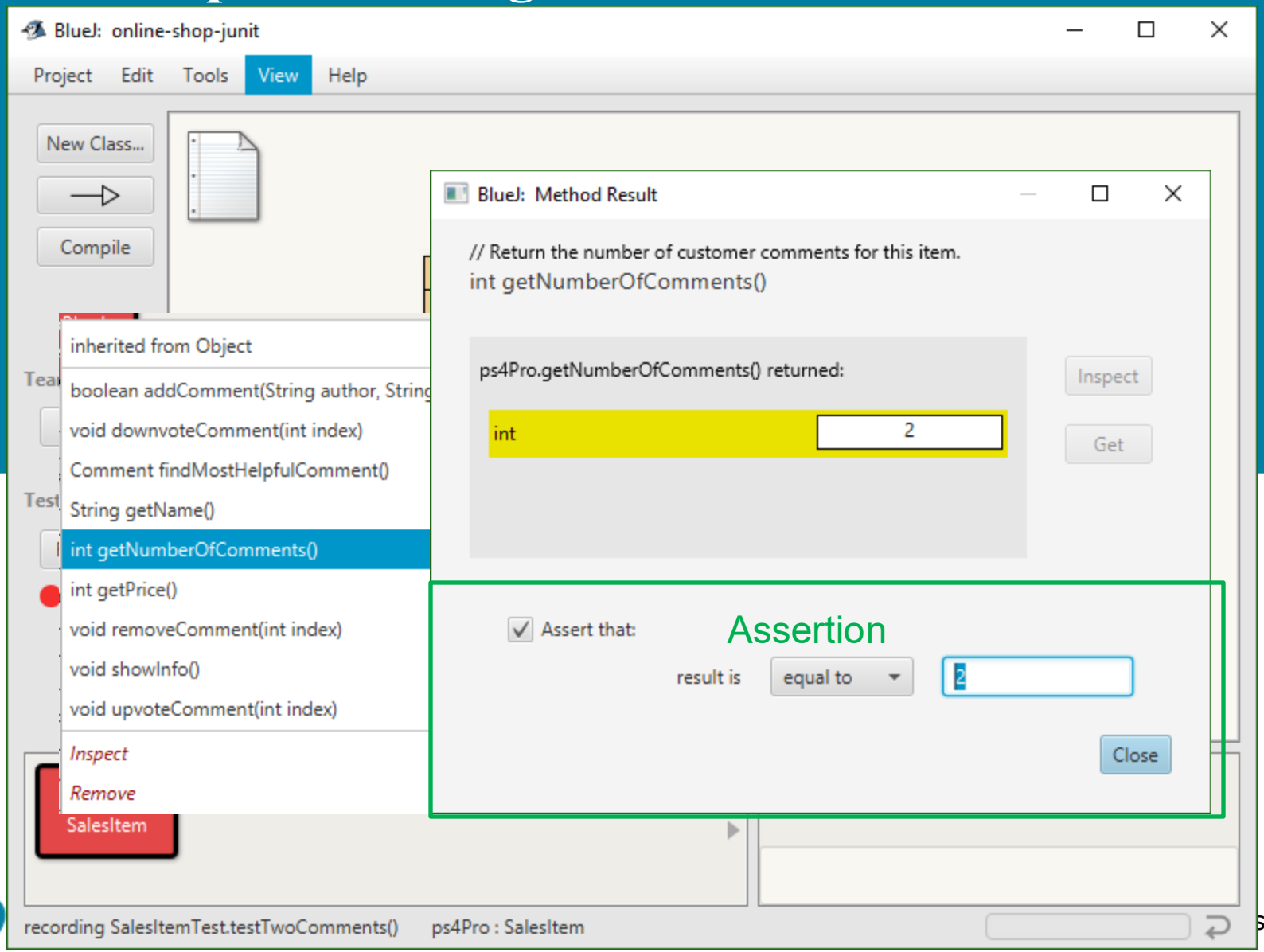
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



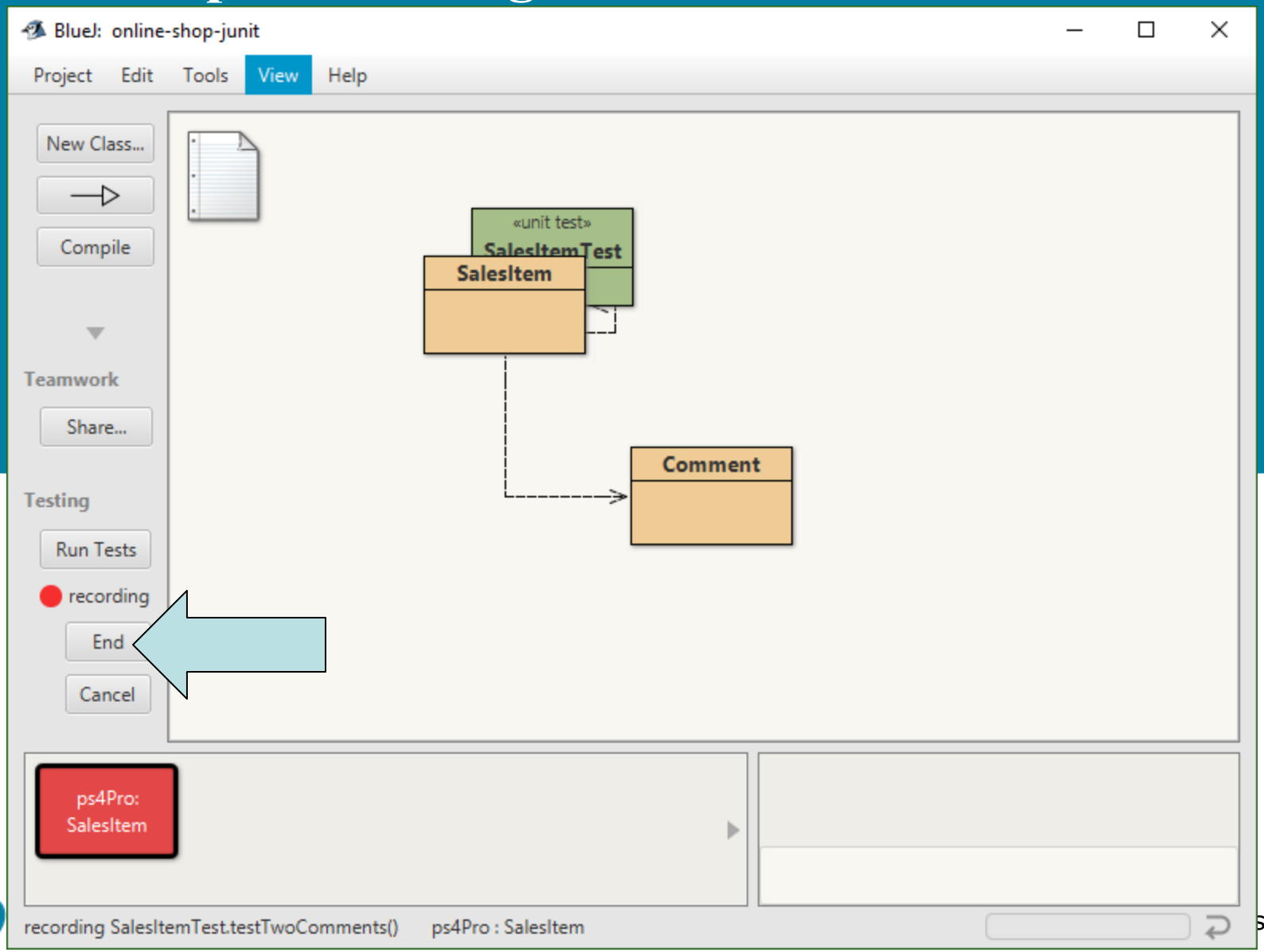
served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación

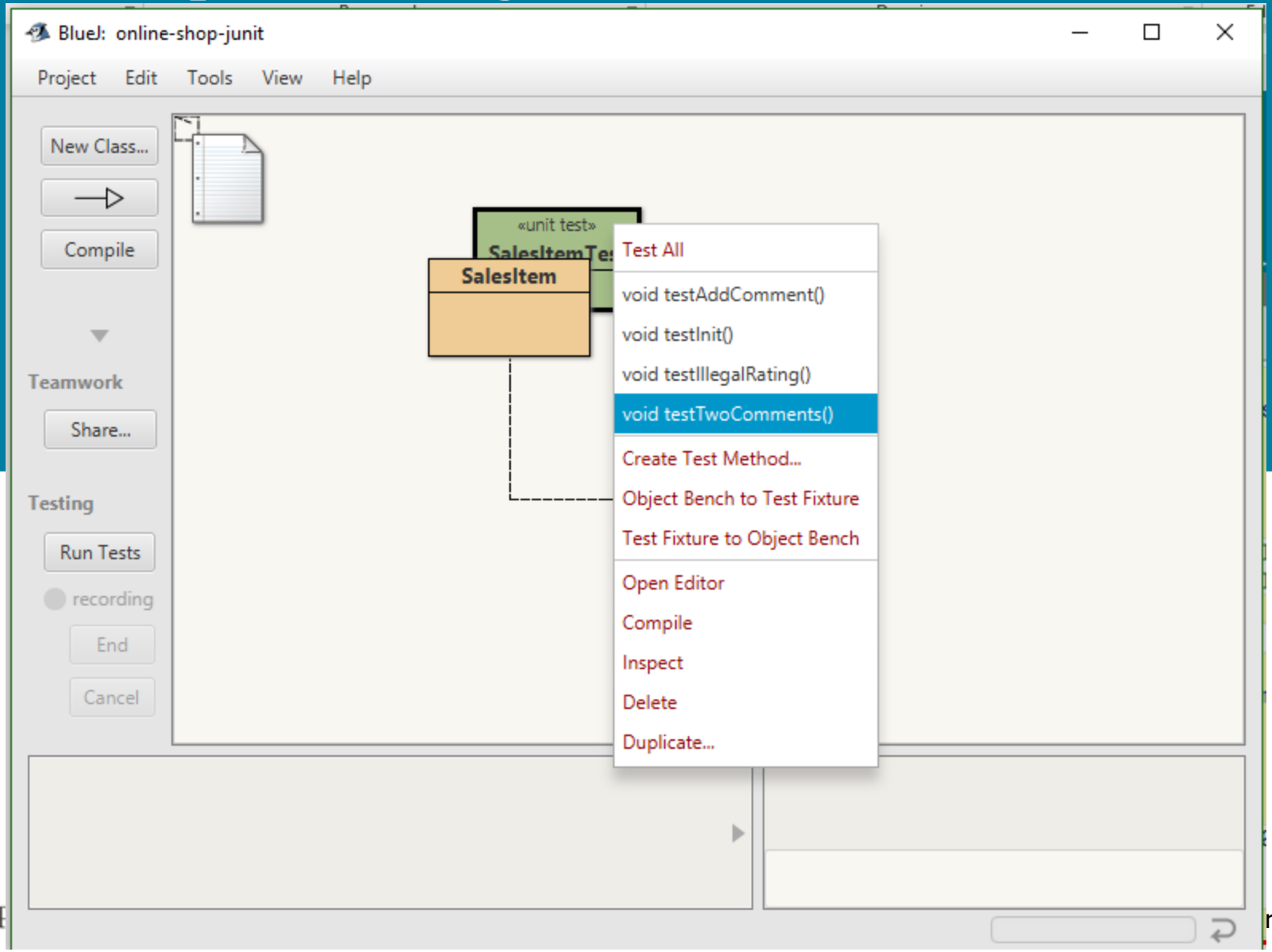
SalesItemTest

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
52      /**
53       * Test that a comment using an illegal rating value is rejected.
54       */
55      @Test
56      public void testIllegalRating()
57      {
58          SalesItem salesIte1 = new SalesItem("Java For Complete Idiots, Vol 2", 19900);
59          assertEquals(false, salesIte1.addComment("Joshua Black", "Not worth the money. The fo
60      }
61
62      /**
63       * Test that a sales item is correctly initialised (name and price).
64       */
65      @Test
66      public void testInit()
67      {
68          SalesItem salesIte1 = new SalesItem("test name", 1000);
69          assertEquals("test name", salesIte1.getName());
70          assertEquals(1000, salesIte1.getPrice());
71      }
72
73      @Test
74      public void testTwoComments()
75      {
76          SalesItem ps4Pro = new SalesItem("PS4 Pro 1 TB", 40000);
77          assertEquals(true, ps4Pro.addComment("Geralt de Rivia", "Amazing stuff", 5));
78          assertEquals(true, ps4Pro.addComment("Yennefer of Vengerberg", "I prefer magic", 2));
79          assertEquals(2, ps4Pro.getNumberOfComments());
80      }
81  }
```

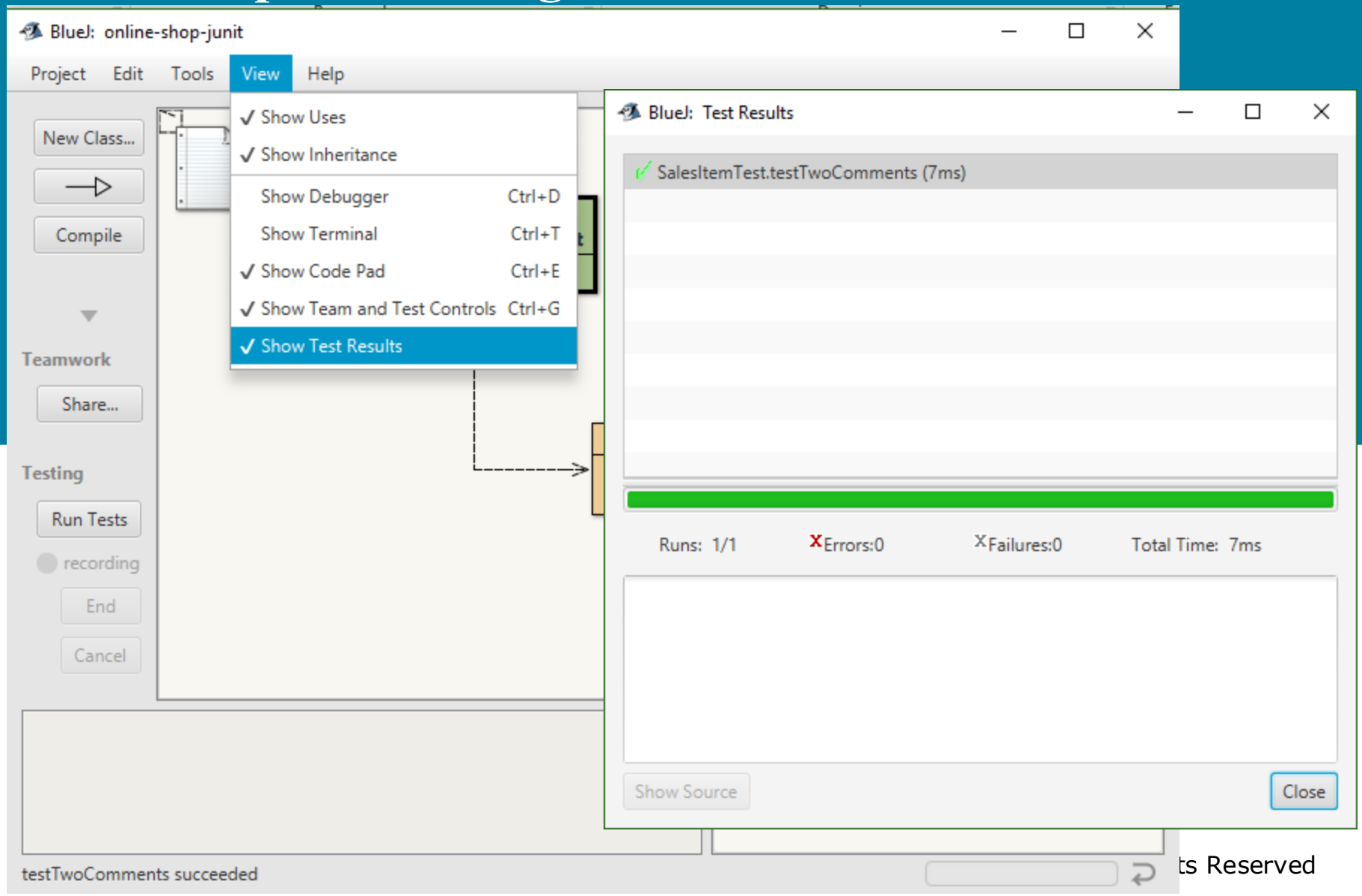
Pea served

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



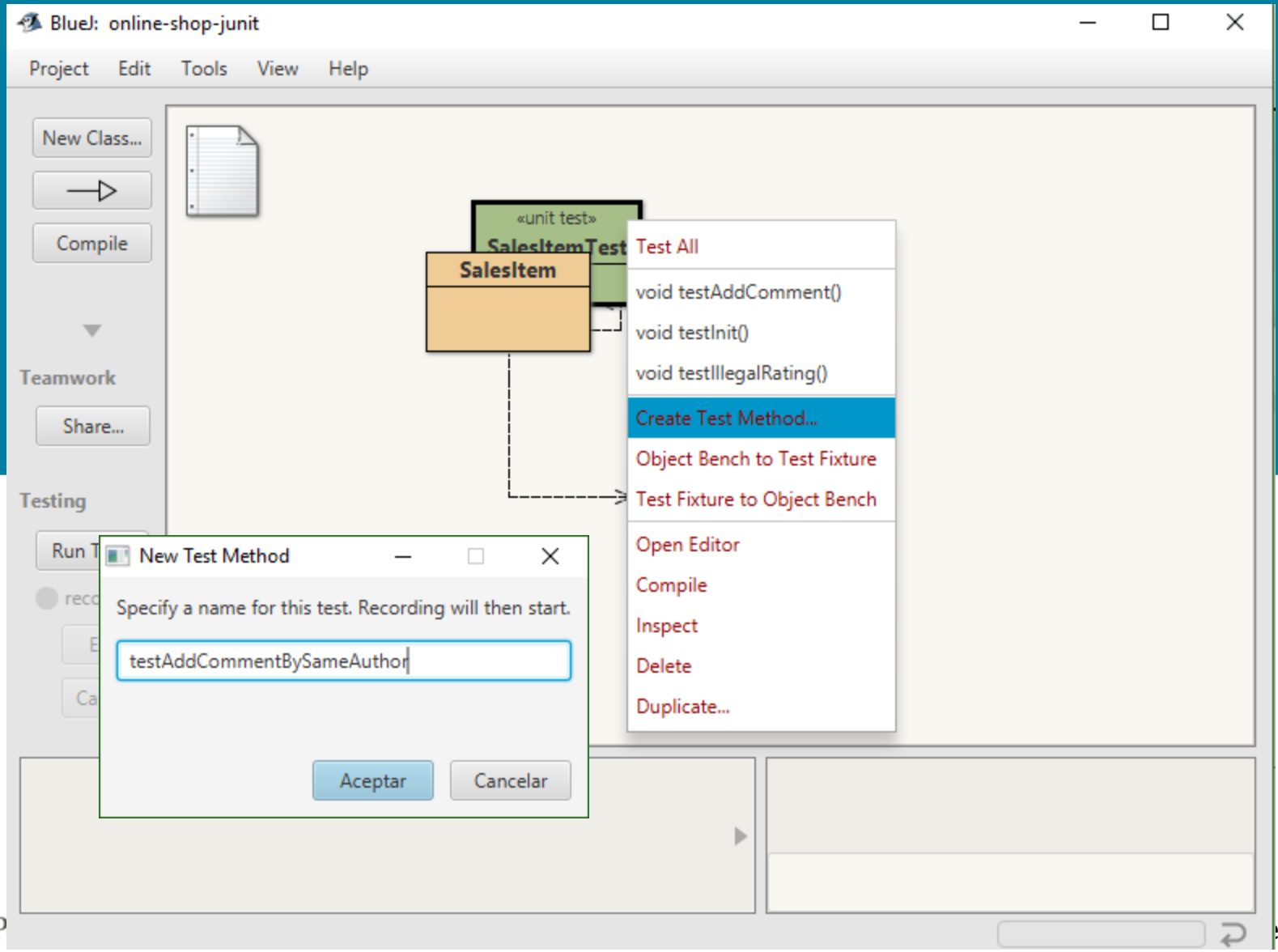
rved

Demo de Online-Shop-Junit – añadiendo un nuevo método de prueba con grabación



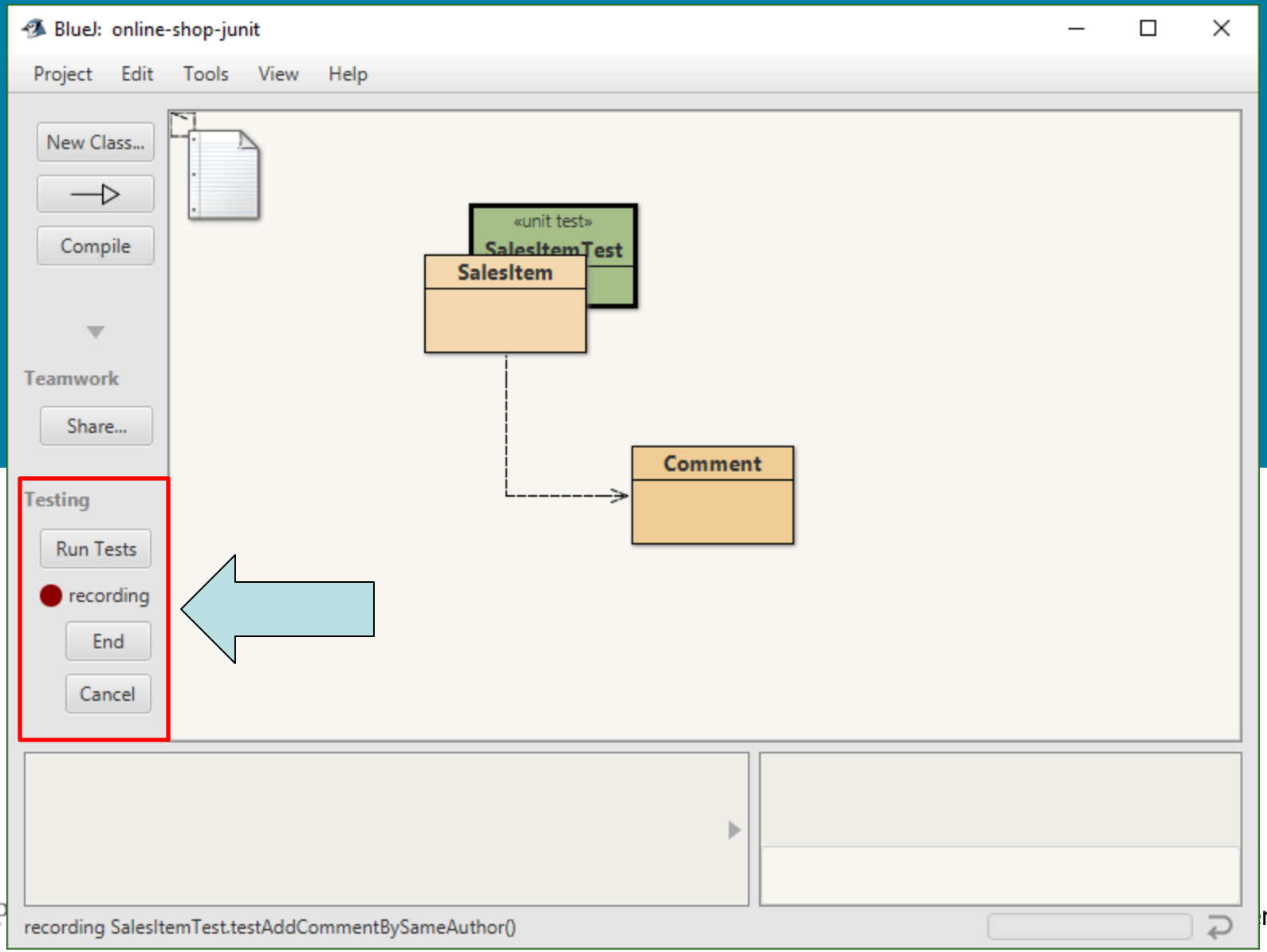
ts Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



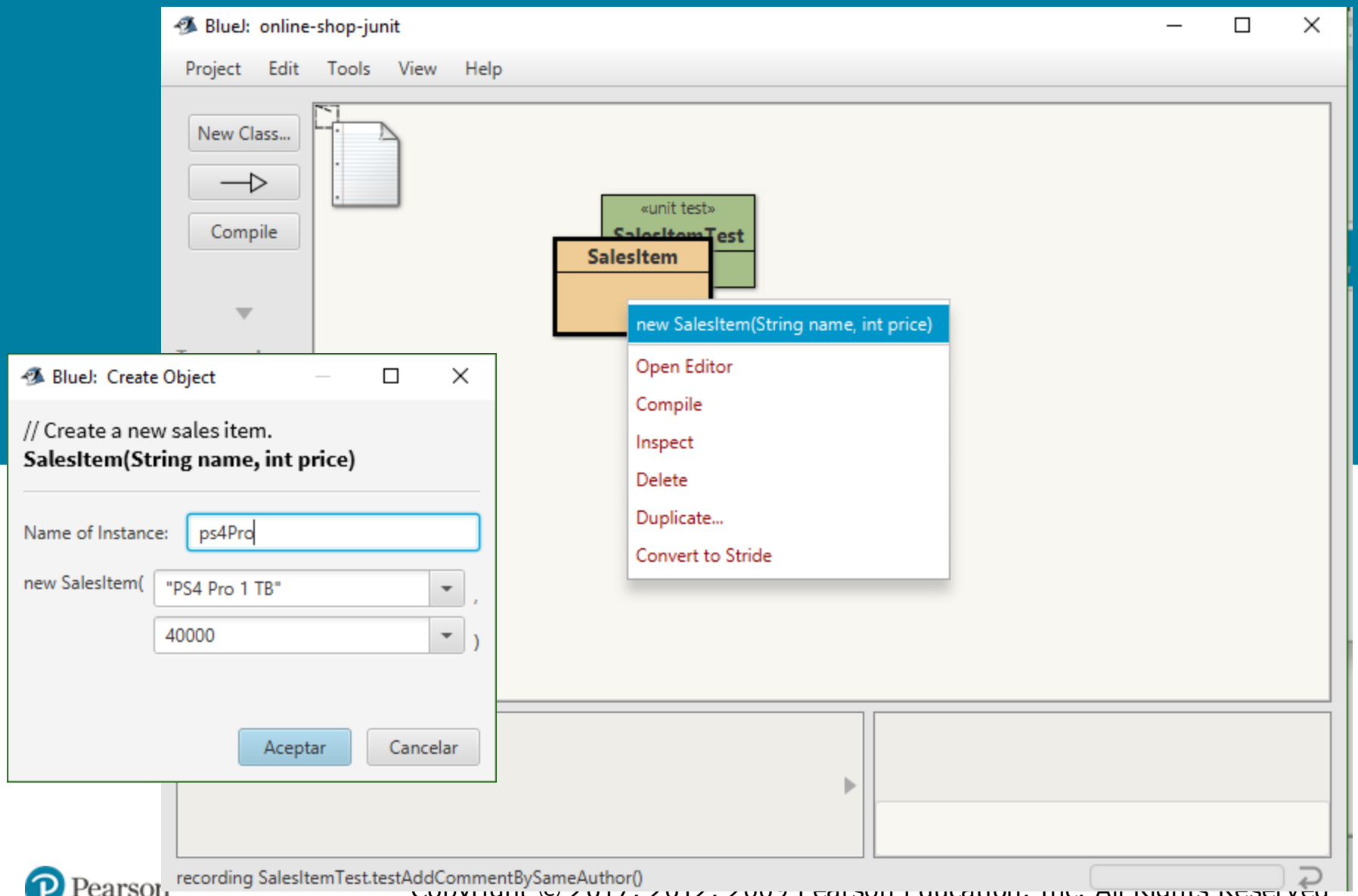
erved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



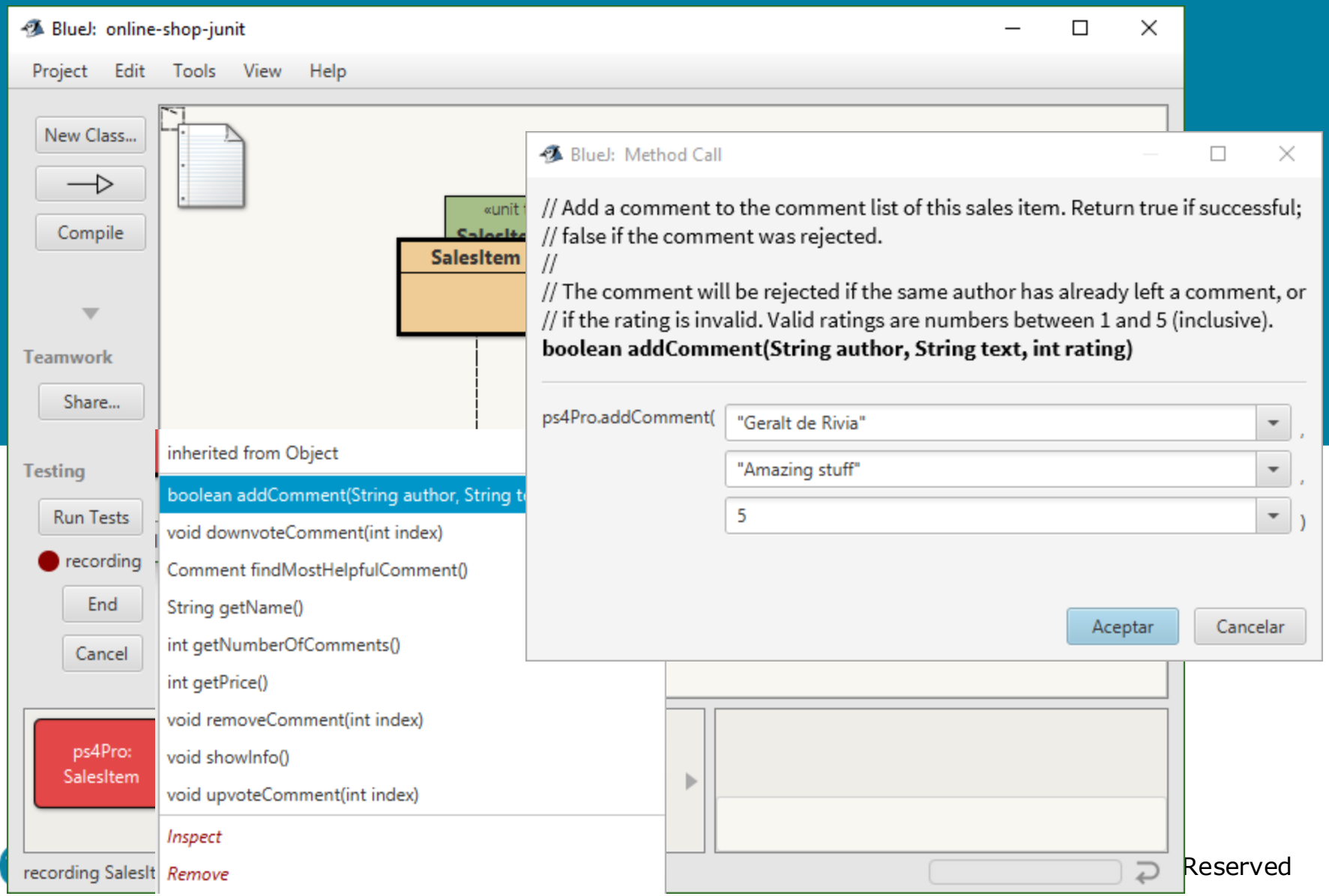
erved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



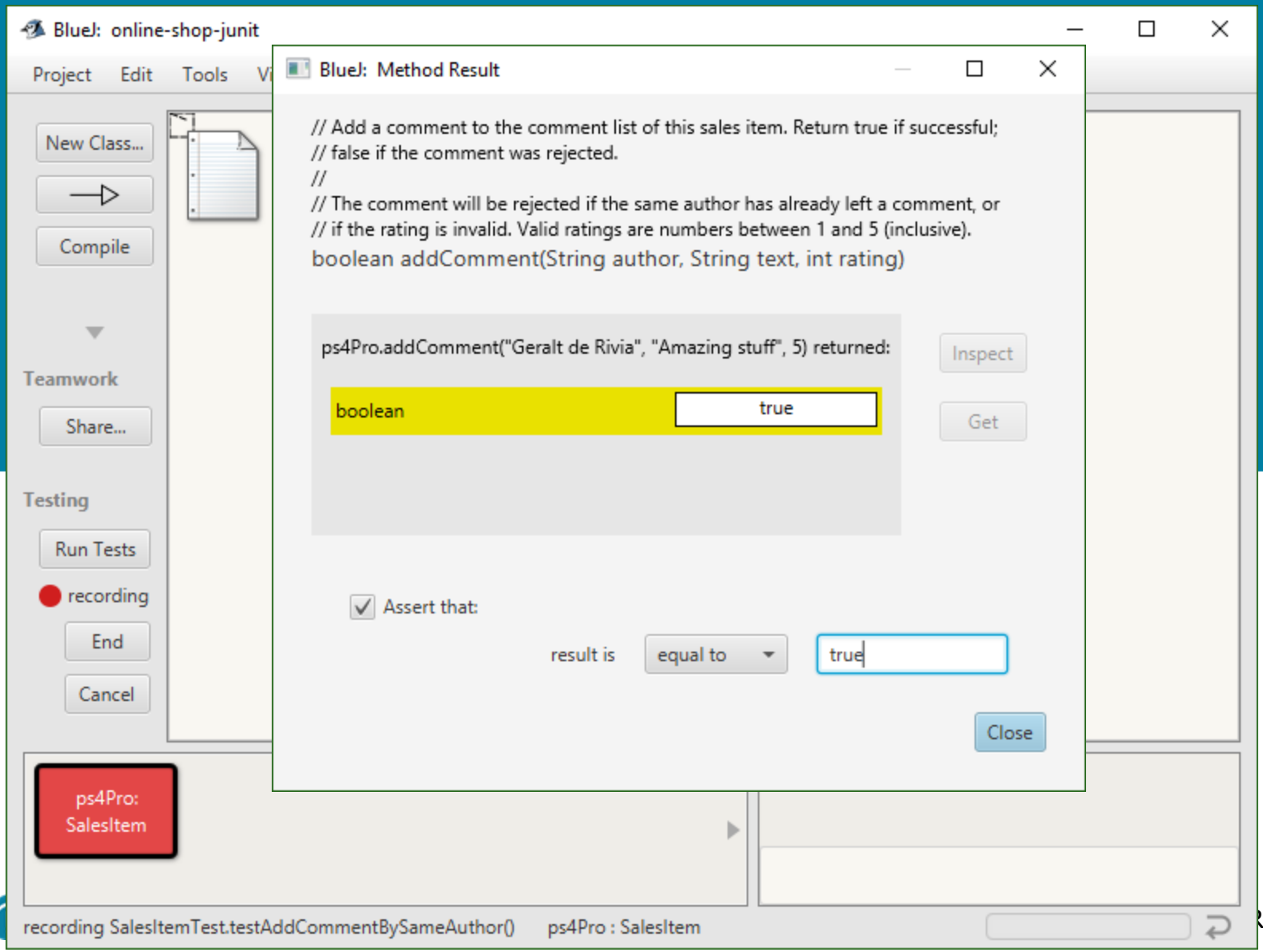
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



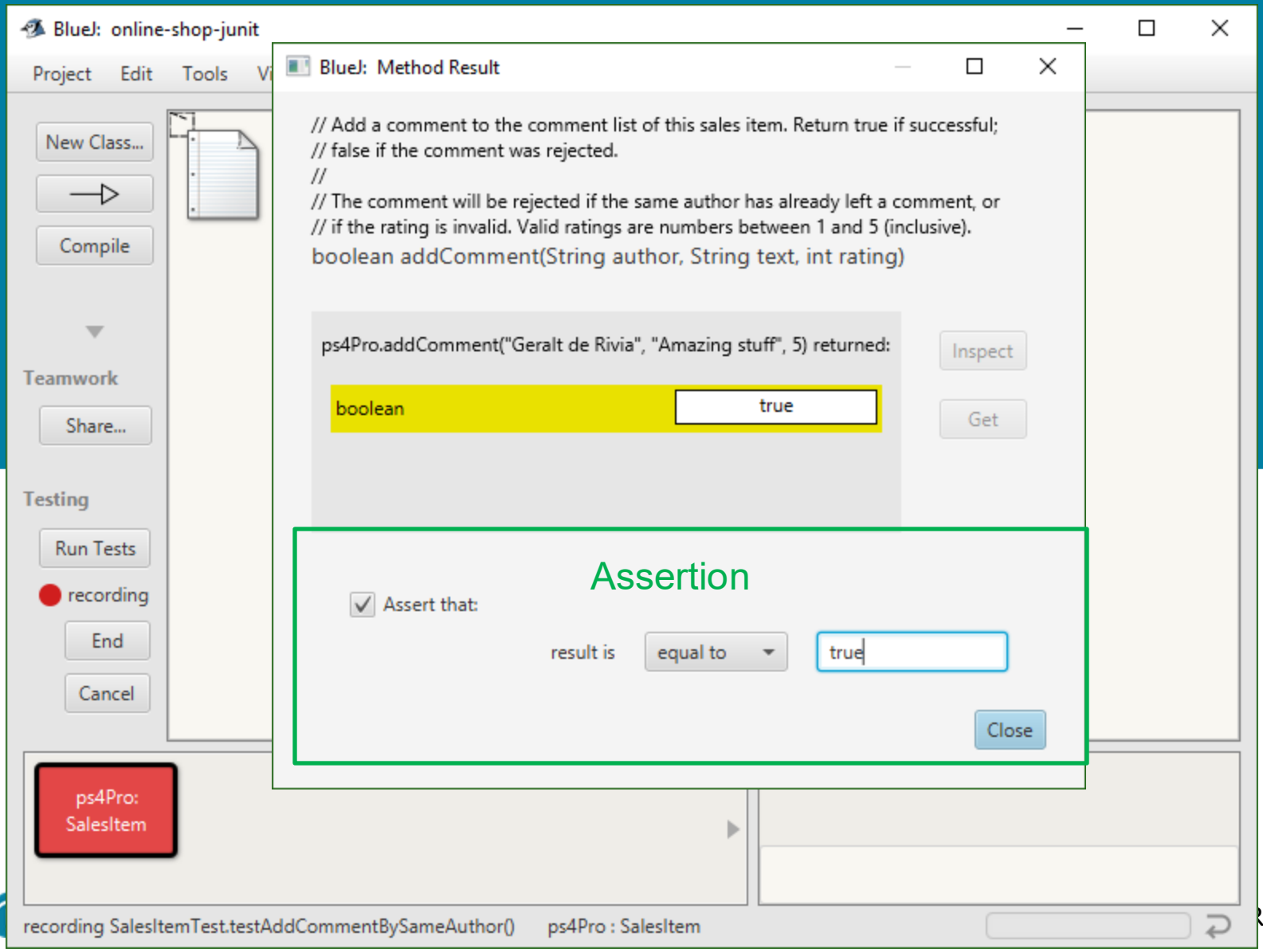
Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



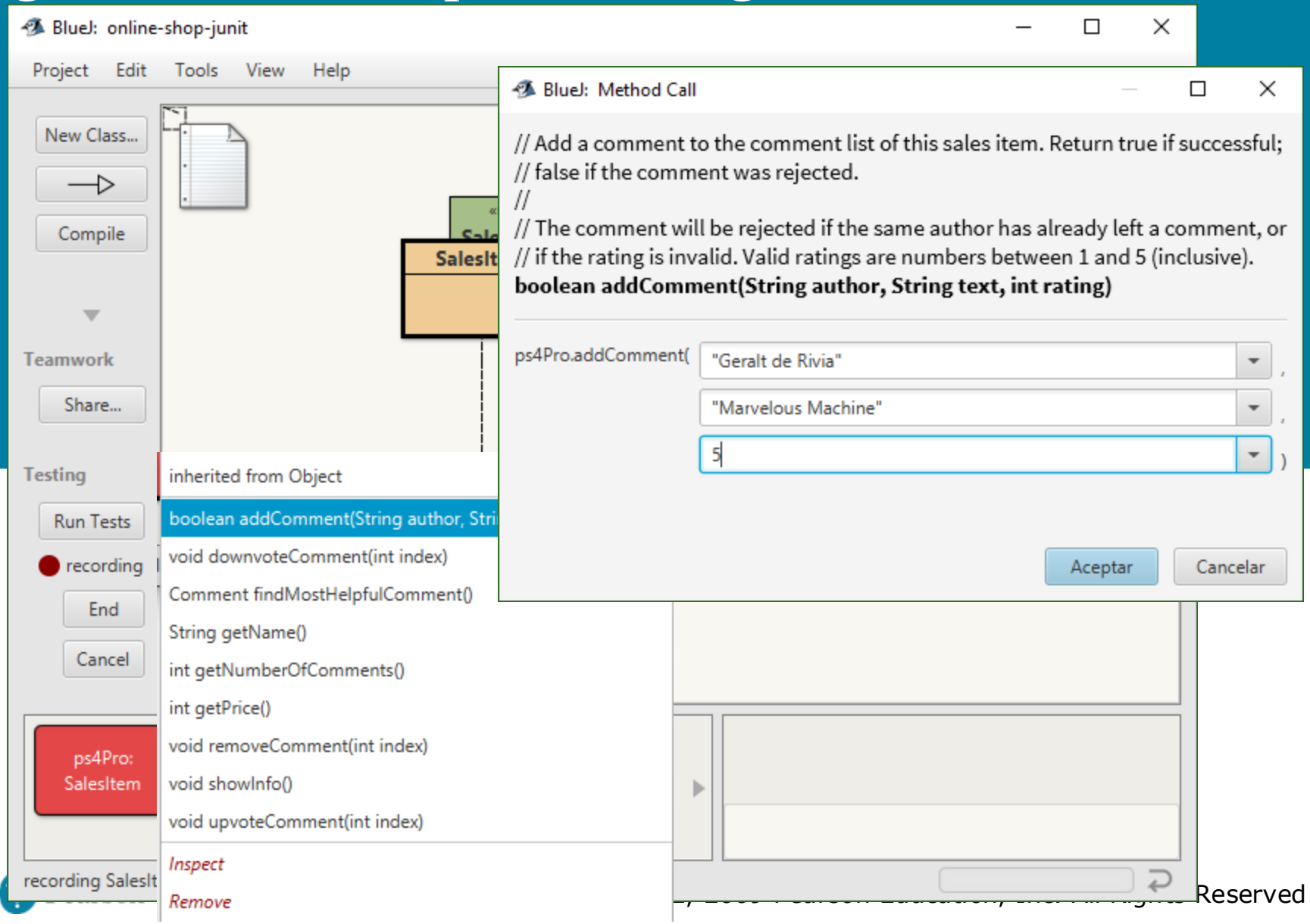
Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



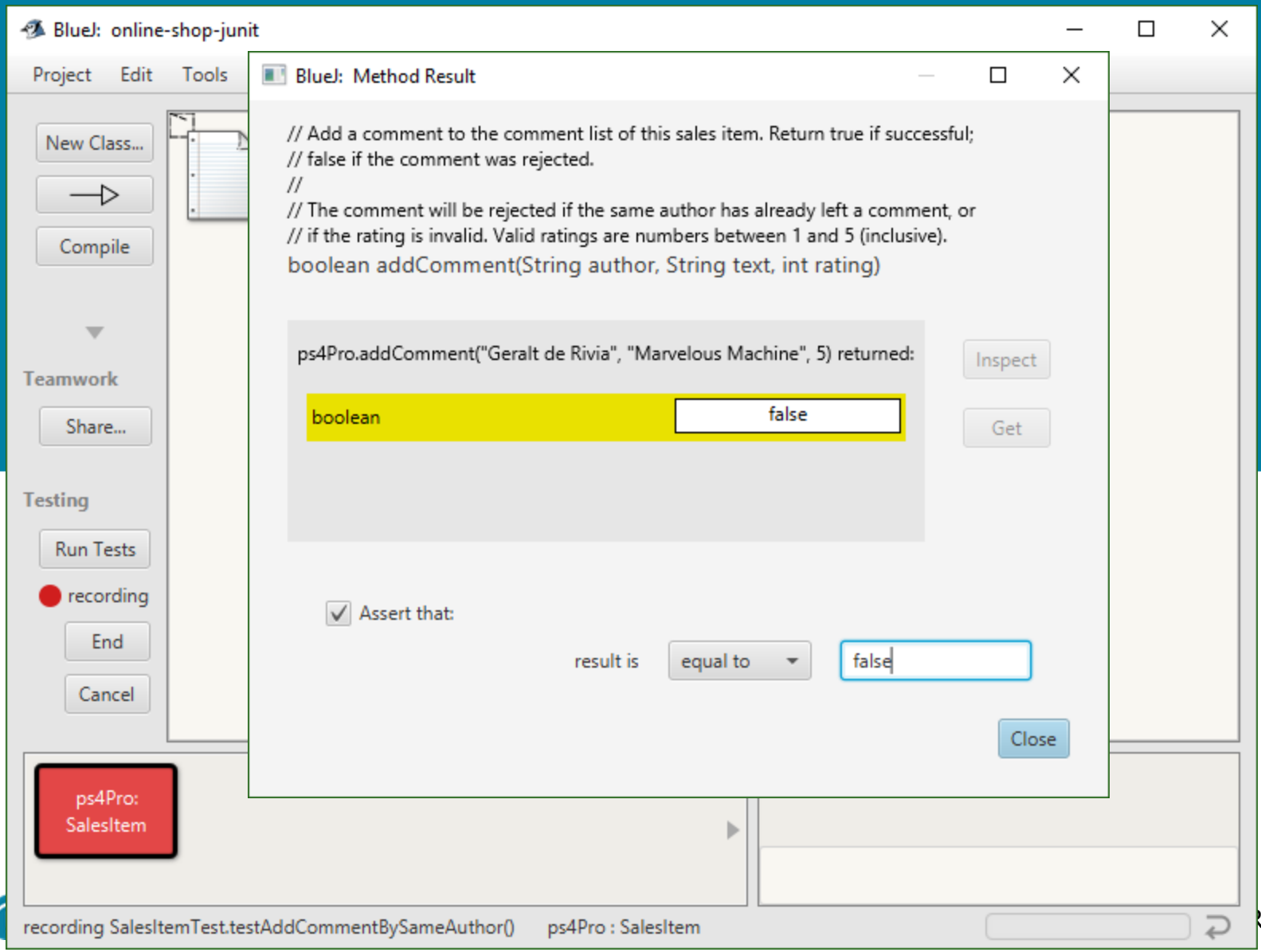
Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



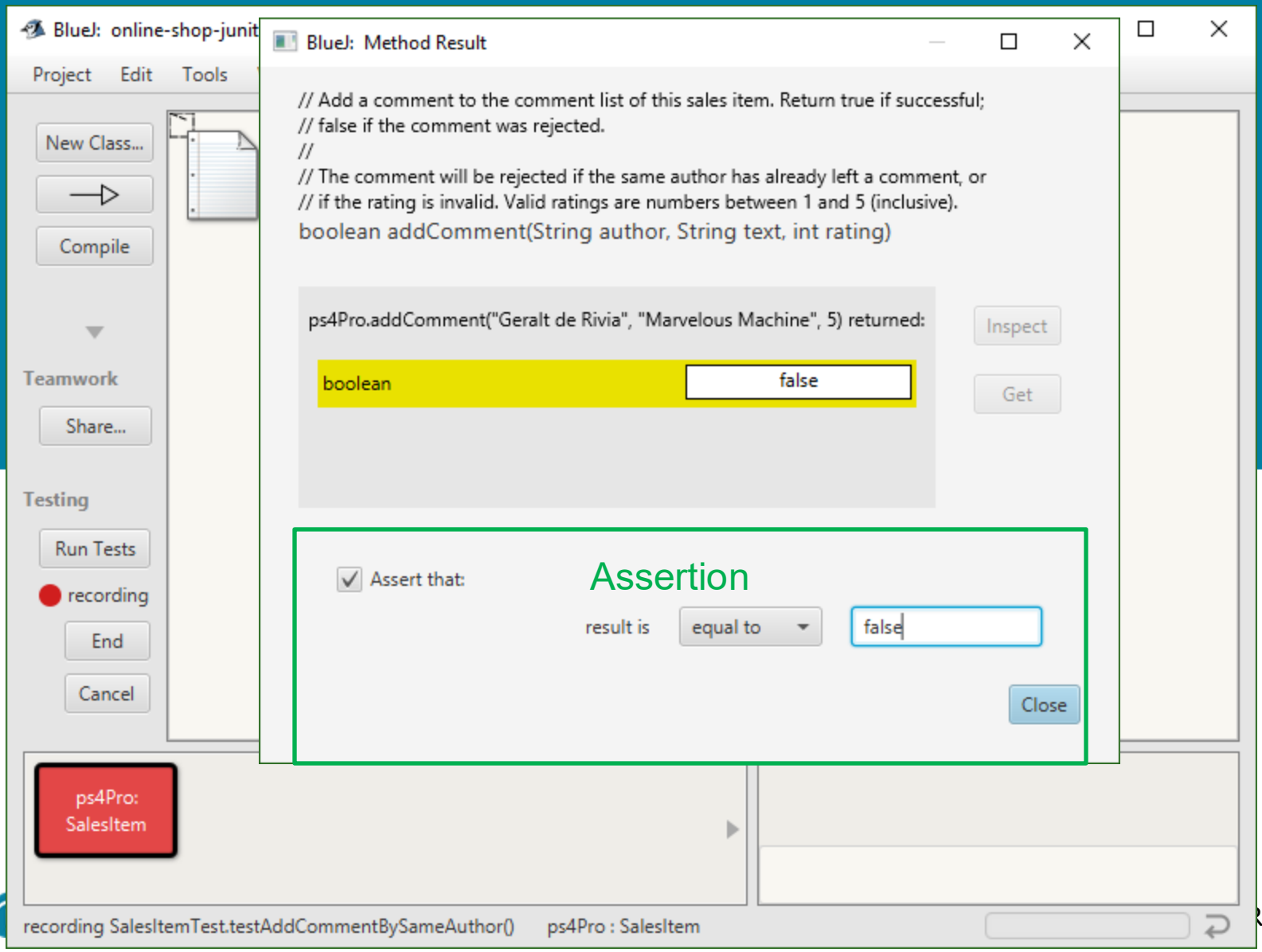
Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



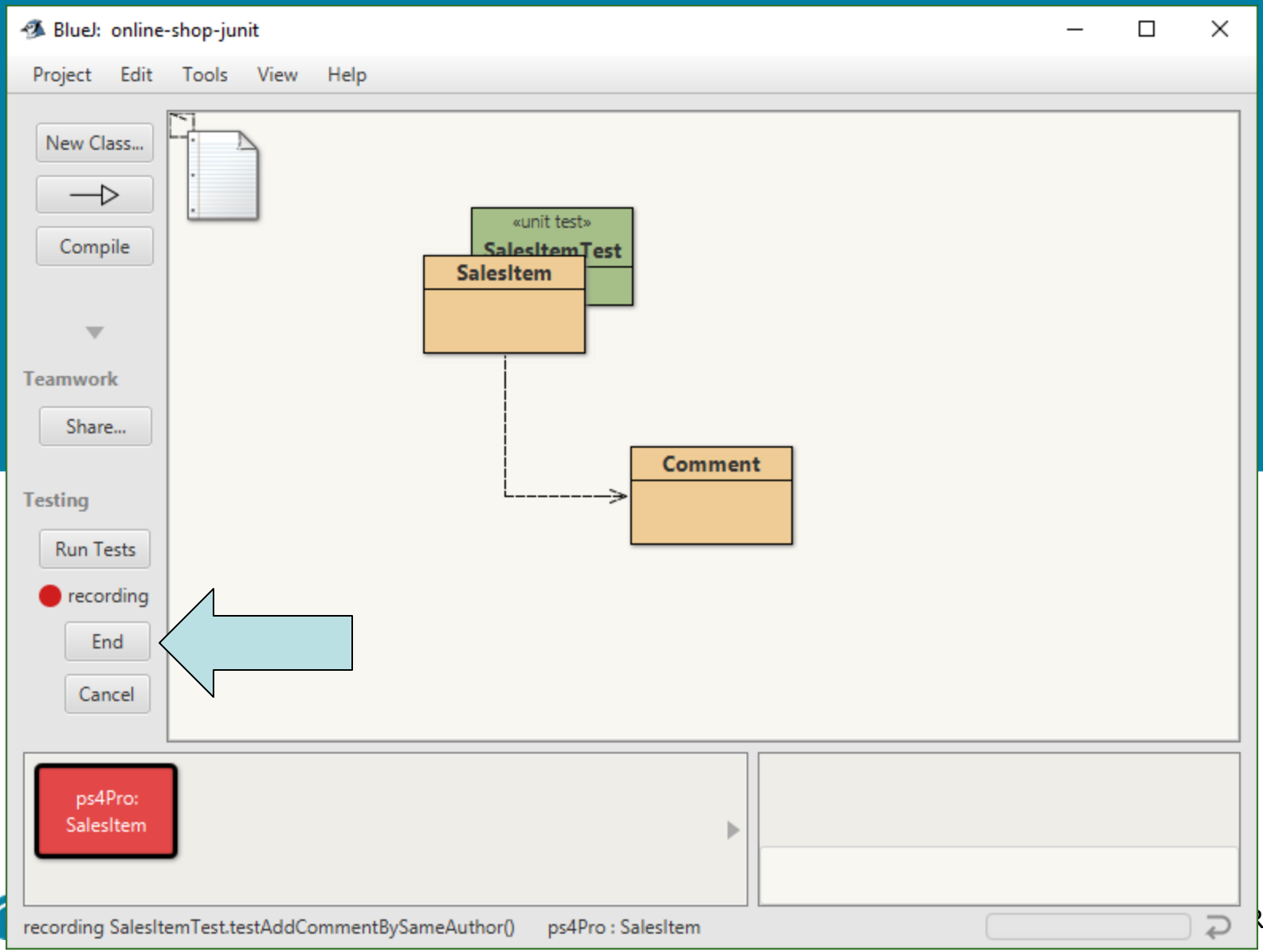
Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación

SalesItemTest

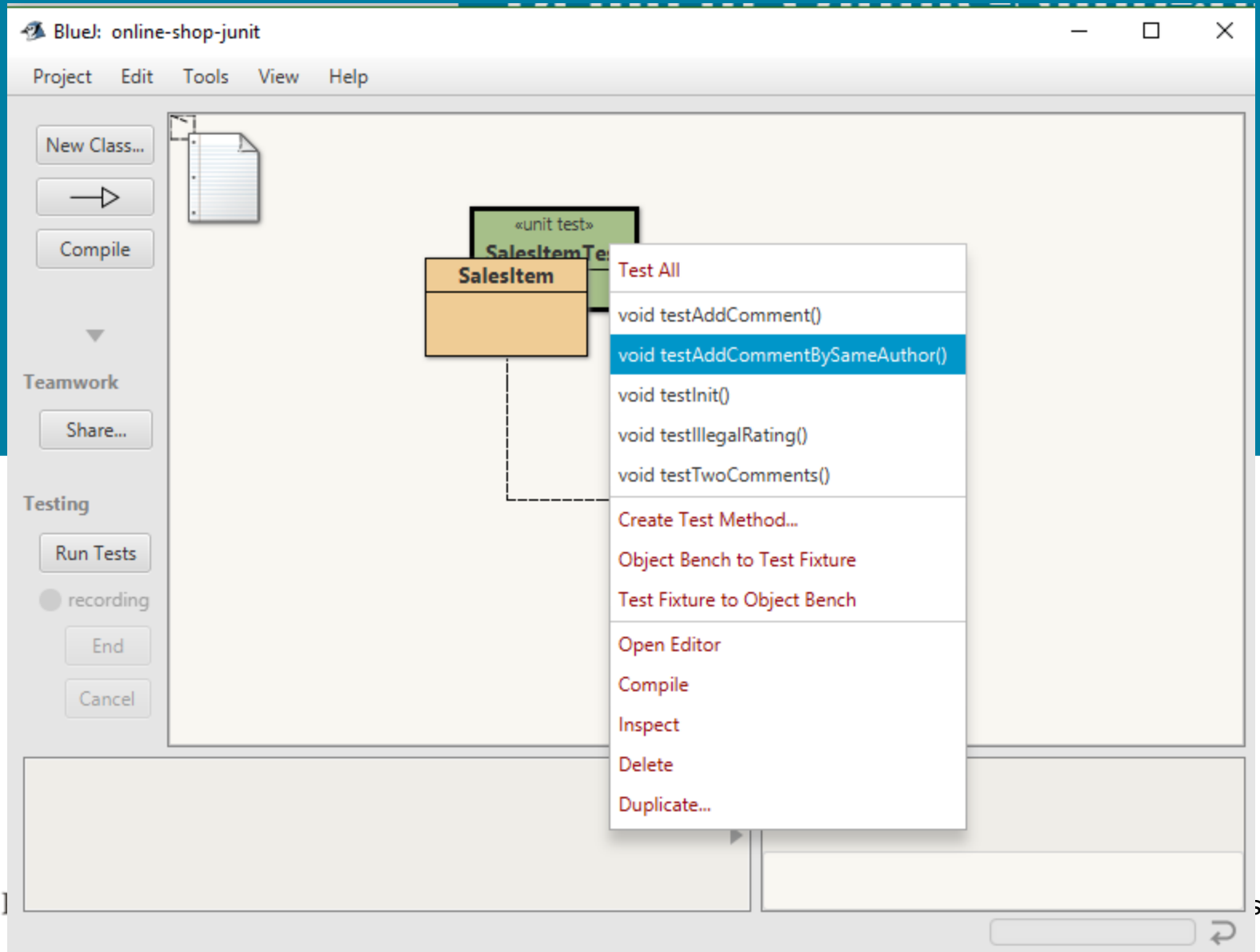
Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
64      */
65     @Test
66     public void testInit()
67     {
68         SalesItem salesIte1 = new SalesItem("test name", 1000);
69         assertEquals("test name", salesIte1.getName());
70         assertEquals(1000, salesIte1.getPrice());
71     }
72
73     @Test
74     public void testTwoComments()
75     {
76         SalesItem ps4Pro = new SalesItem("PS4 Pro 1 TB", 40000);
77         assertEquals(true, ps4Pro.addComment("Geralt de Rivia", "Amazing stuff", 5));
78         assertEquals(true, ps4Pro.addComment("Yennefer of Vengerberg", "I prefer magic", 2));
79         assertEquals(2, ps4Pro.getNumberOfComments());
80     }
81
82     @Test
83     public void testAddCommentBySameAuthor()
84     {
85         SalesItem ps4Pro = new SalesItem("PS4 Pro 1TB", 40000);
86         assertEquals(true, ps4Pro.addComment("Geralt de Rivia", "Amazing stuff", 5));
87         assertEquals(false, ps4Pro.addComment("Geralt de Rivia", "Marvelous Machine", 5));
88     }
89 }
```

Pearson

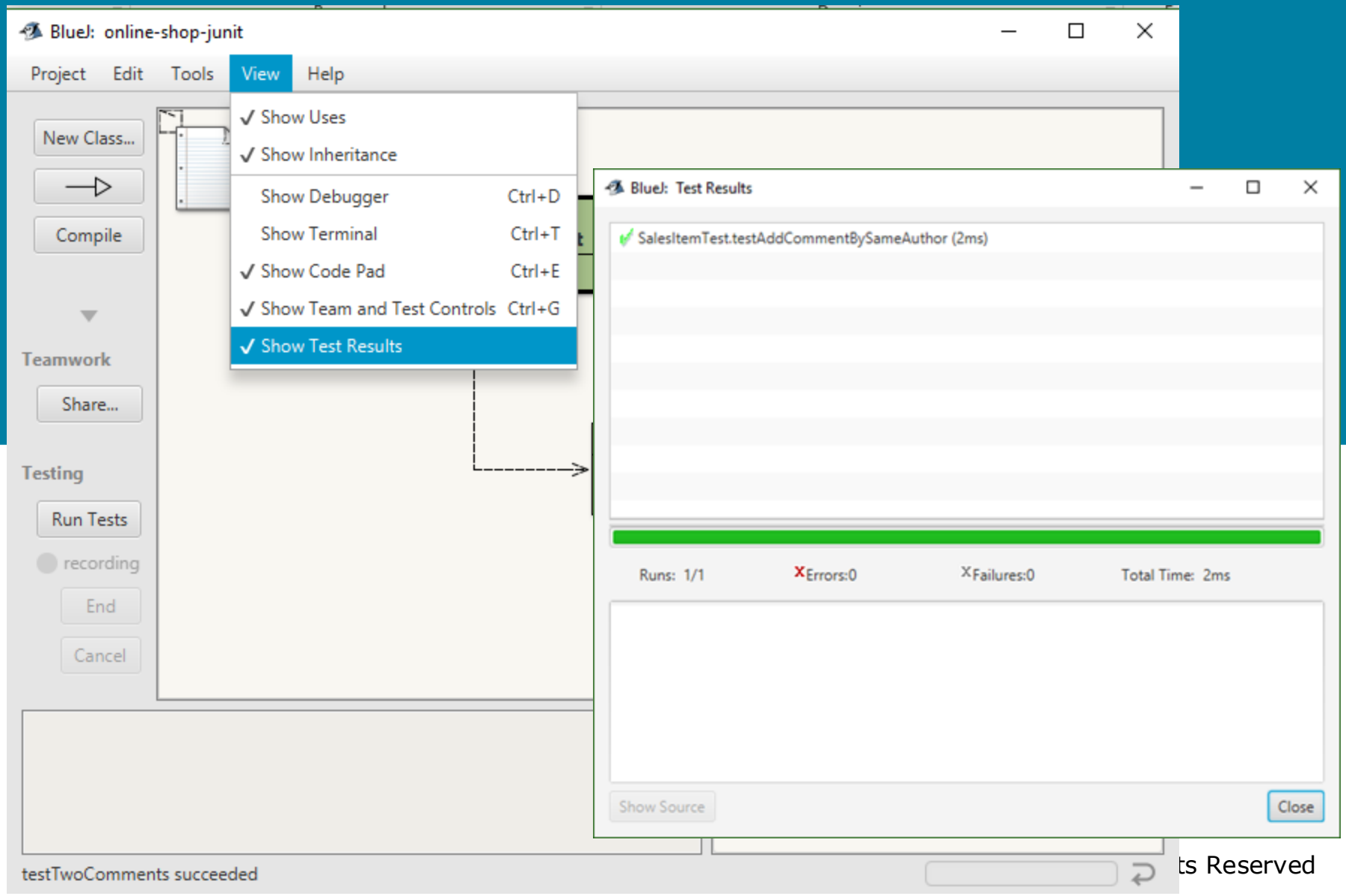
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



served

Demo de Online-Shop-Junit – añadiendo un segundo método de prueba con grabación



ts Reserved

Demo de Online-Shop-Junit

— cambiando el código fuente de la clase original y probando con el segundo método de prueba

SalesItem

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

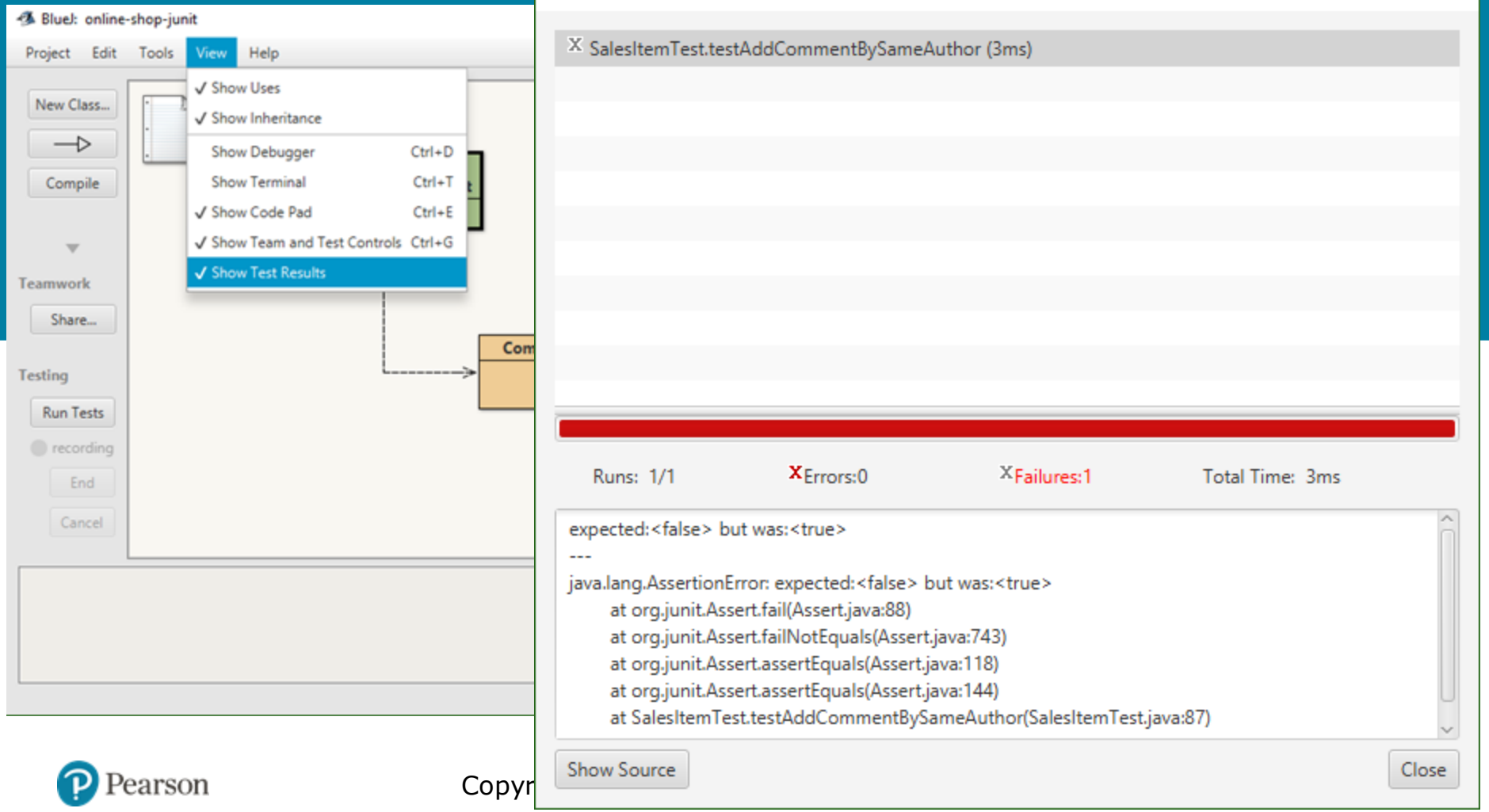
```
45      }
46
47      /**
48       * Return the number of customer comments for this item.
49       */
50      public int getNumberOfComments()
51      {
52          return comments.size();
53      }
54
55      /**
56       * Add a comment to the comment list of this sales item. Return true if successful;
57       * false if the comment was rejected.
58       *
59       * The comment will be rejected if the same author has already left a comment, or
60       * if the rating is invalid. Valid ratings are numbers between 1 and 5 (inclusive).
61       */
62      public boolean addComment(String author, String text, int rating)
63      {
64          if(ratingInvalid(rating)) {  // reject invalid ratings
65              return false;
66          }
67          /*
68          if(findCommentByAuthor(author) != null) {  // reject mutiple comments by same author
69              return false;
70          }
71          */
72          comments.add(new Comment(author, text, rating));
73          return true;
74      }
75
```

Pearson

Rights Reserved

Demo de Online-Shop-Junit

– cambiando el código fuente de la clase original y probando con el segundo método de prueba

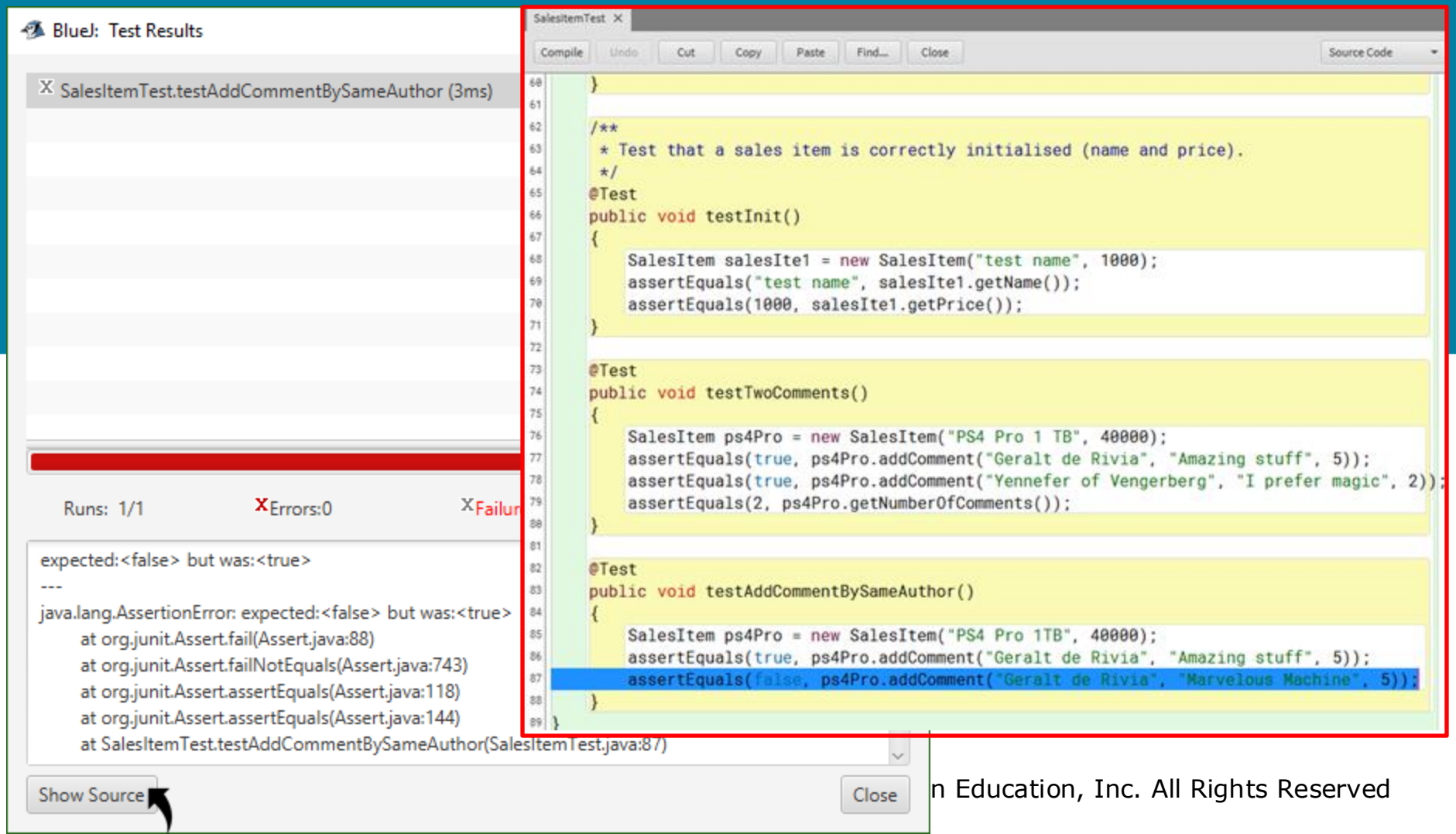


Pearson

Copyr

Demo de Online-Shop-Junit

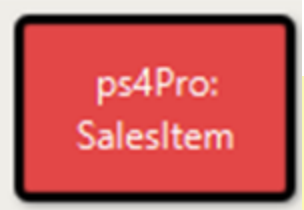
– cambiando el código fuente de la clase original y probando con el segundo método de prueba



n Education, Inc. All Rights Reserved

Fixtures

- A medida que se crean métodos de prueba, es común crear objetos similares para cada uno de ellos.
  - Por ejemplo, cada prueba de la clase *SalesItem* implicará crear al menos un *SalesItem* e inicializarlo, a menudo añadiendo uno o más comentarios.




```
@Test
public void testAddCommentBySameAuthor()
{
    SalesItem ps4Pro = new SalesItem("PS4 Pro 1TB", 40000);
    assertEquals(true, ps4Pro.addComment("Geralt de Rivia", "Amazing stuff", 5));
    assertEquals(false, ps4Pro.addComment("Geralt de Rivia", "Marvelous Machine", 5));
}
```

```
@Test
public void testTwoComments()
{
    SalesItem ps4Pro = new SalesItem("PS4 Pro 1 TB", 40000);
    assertEquals(true, ps4Pro.addComment("Geralt de Rivia", "Amazing stuff", 5));
    assertEquals(true, ps4Pro.addComment("Yennefer of Vengerberg", "I prefer magic", 2));
    assertEquals(2, ps4Pro.getNumberOfComments());
}
```

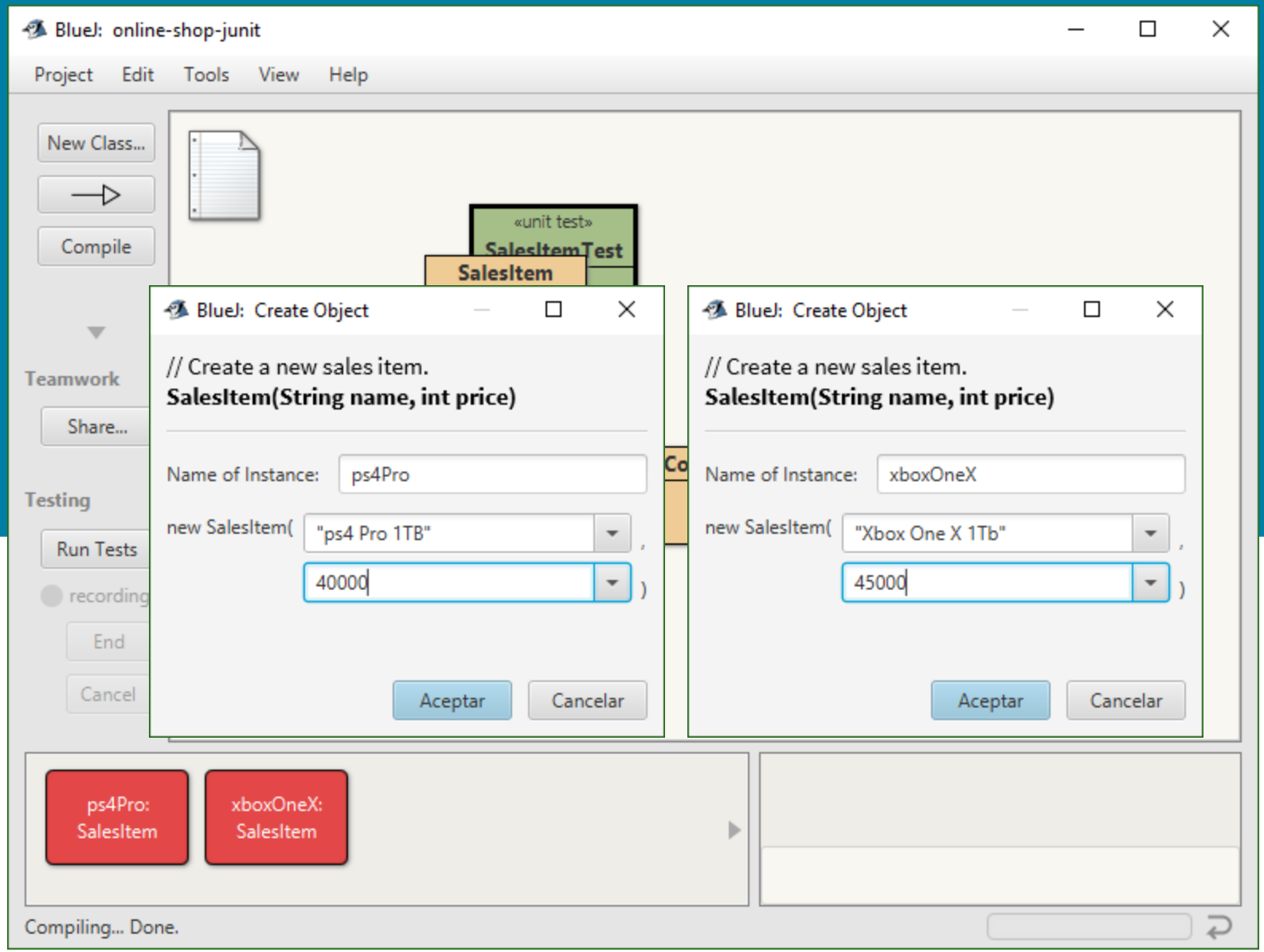
Fixtures

- Un objeto o grupo de objetos que se usan en más de una prueba se conoce como un "fixture".
  - Dos elementos asociados a una clase de prueba nos permite trabajar con fixtures en BlueJ:
    - Desde el banco de objetos a pruebas *Fixture*
    - Desde pruebas *Fixture* al banco de objetos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

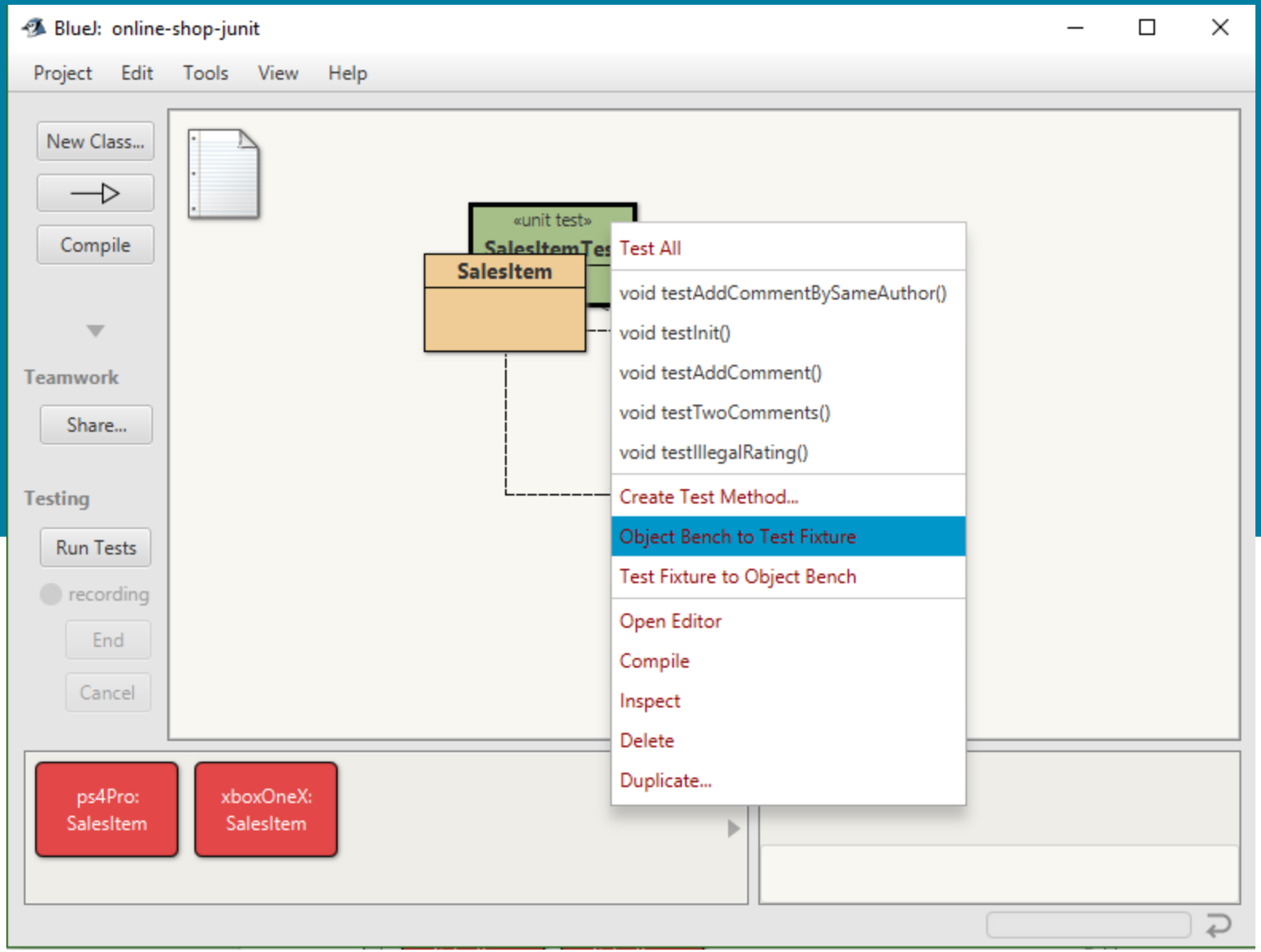
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

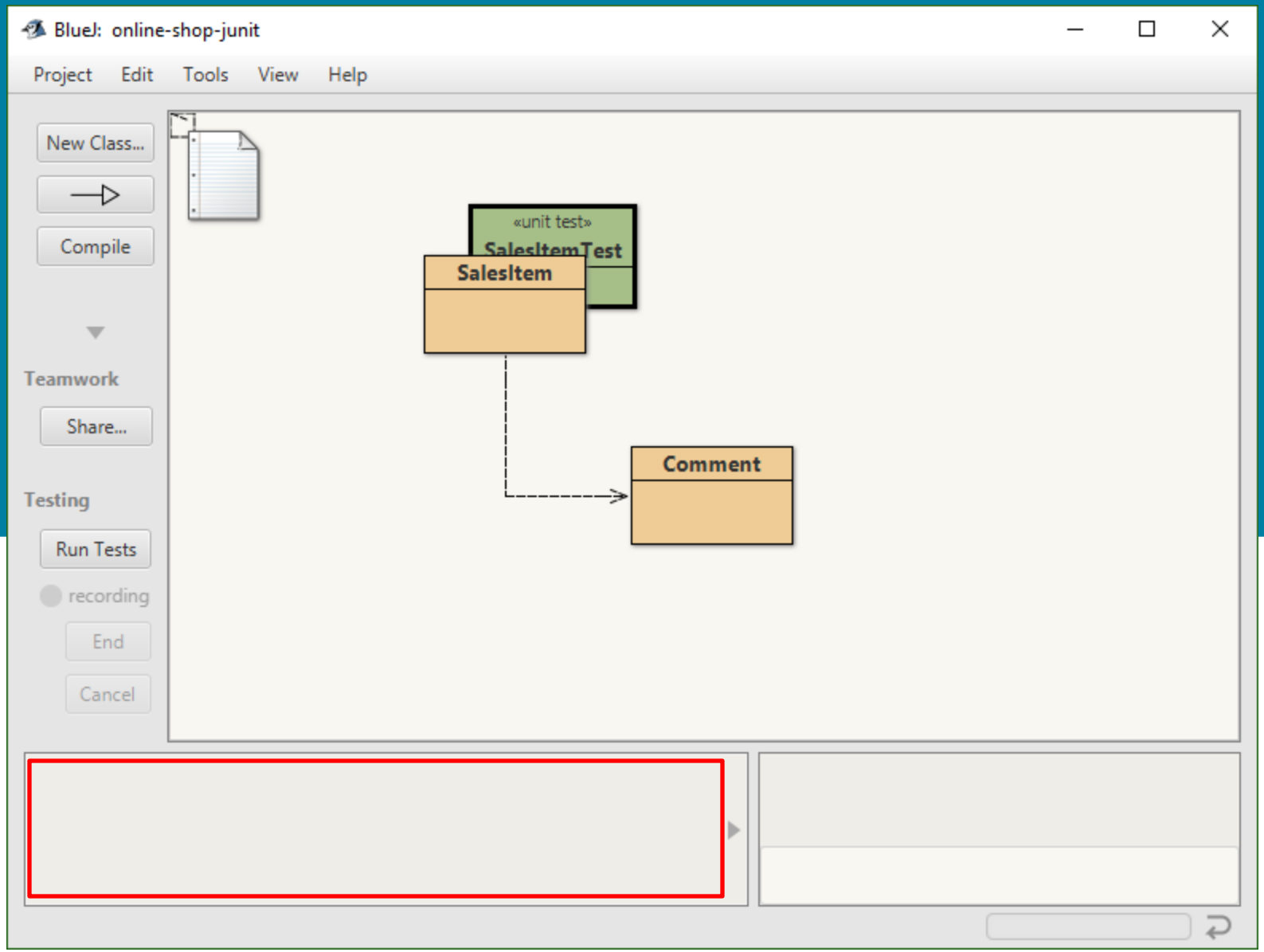
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*

SalesItemTest ×

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

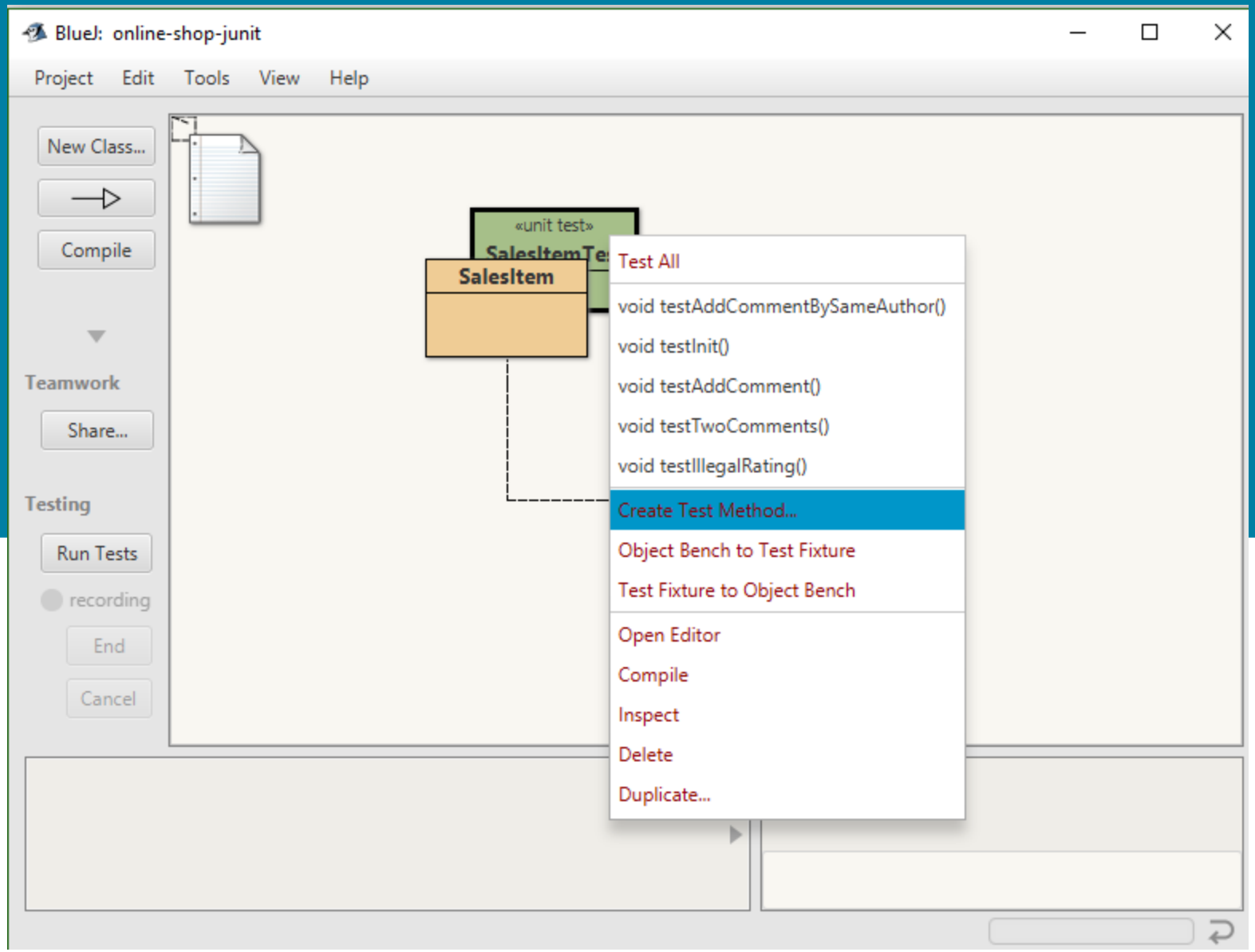
```
/**
 * The test class SalesItemTest.
 *
 * @author  mik
 * @version 0.1
 */
public class SalesItemTest
{
    private SalesItem ps4Pro;
    private SalesItem xboxOneX;

    /**
     * Default constructor for test class SalesItemTest
     */
    public SalesItemTest()
    {
    }

    /**
     * Sets up the test fixture.
     *
     * Called before every test case method.
     */
    @Before
    public void setUp()
    {
        ps4Pro = new SalesItem("ps4 Pro 1TB", 40000);
        xboxOneX = new SalesItem("Xbox One X 1Tb", 45000);
    }
```

Pearson Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

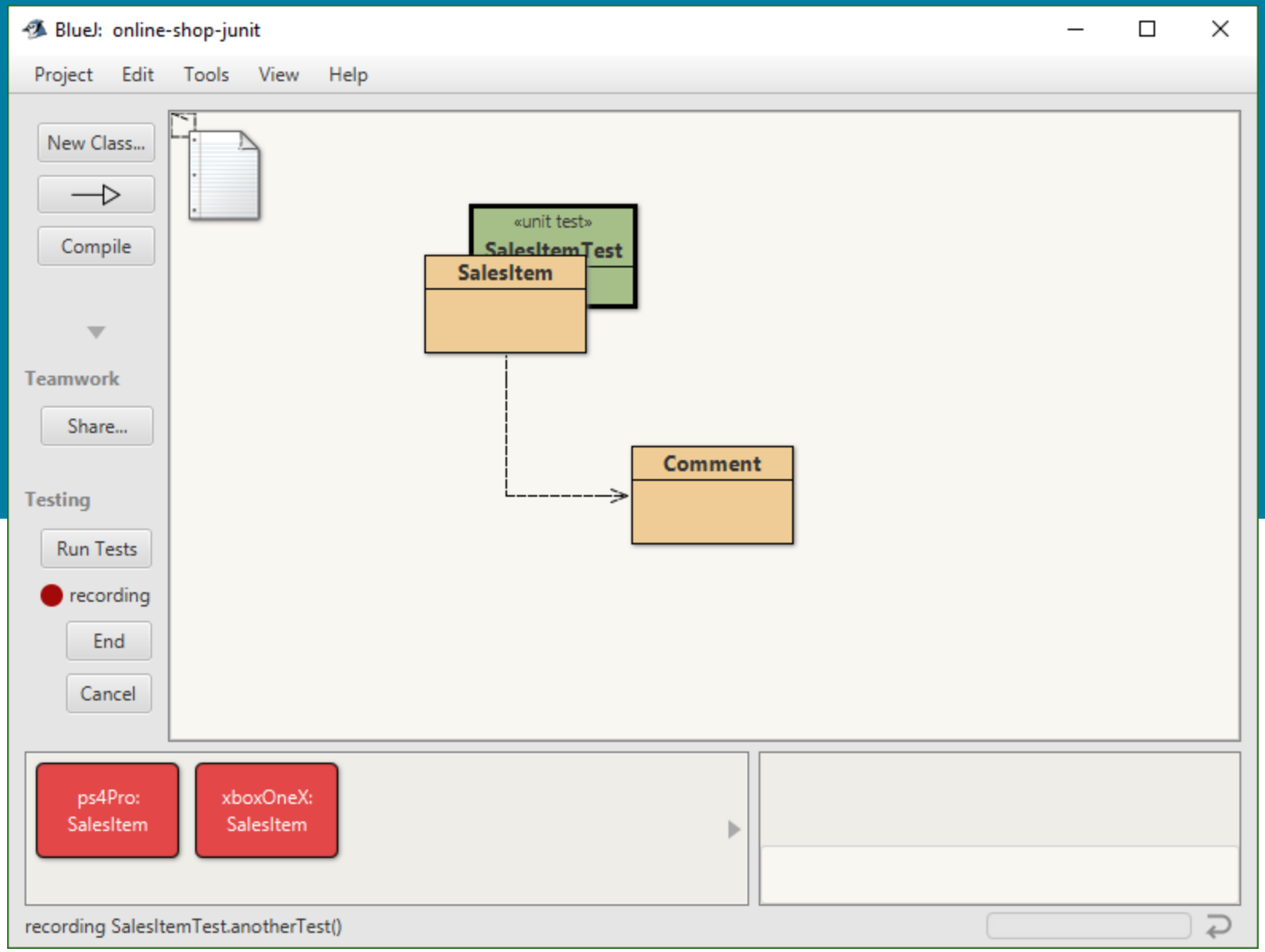
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

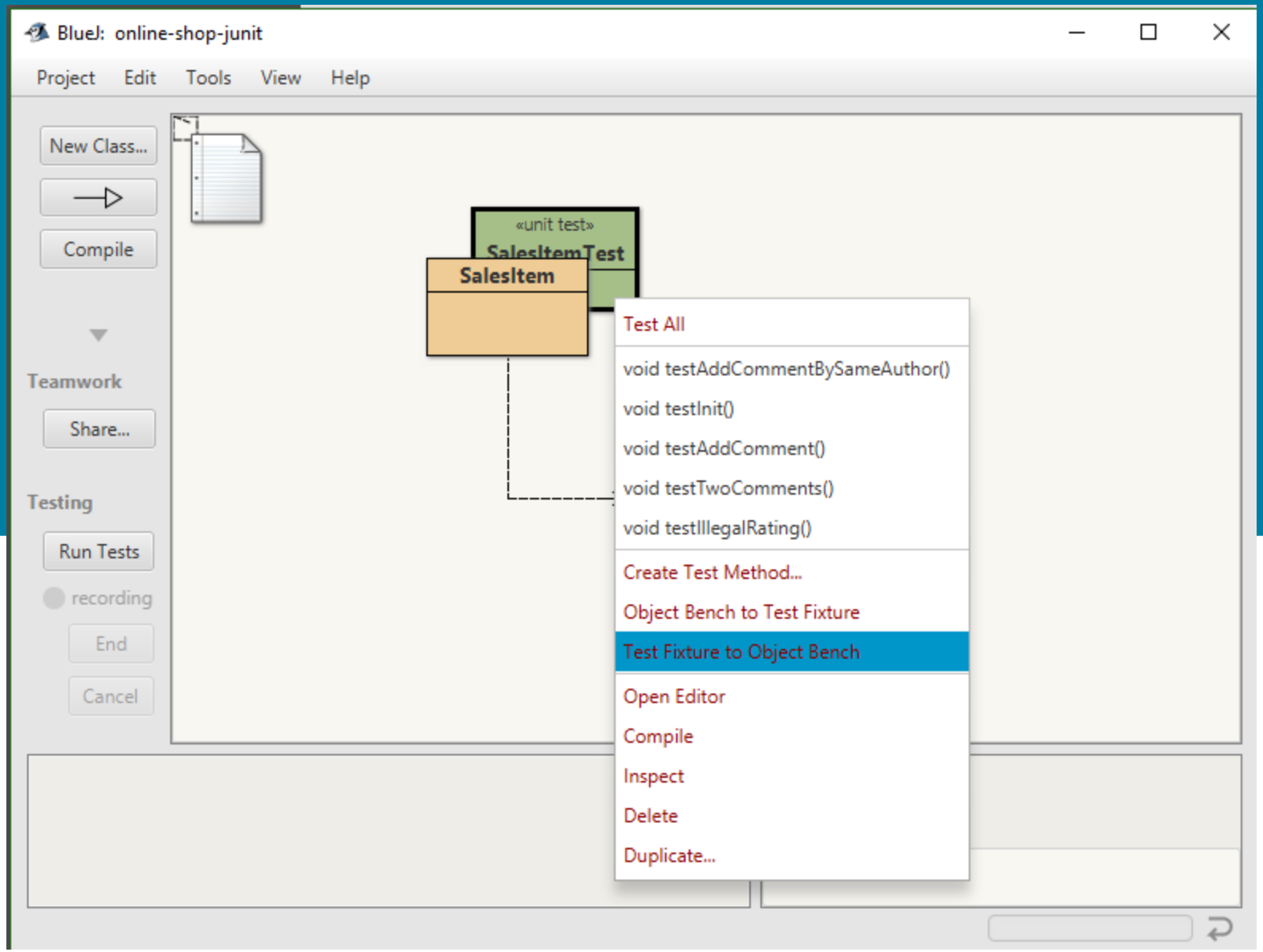
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

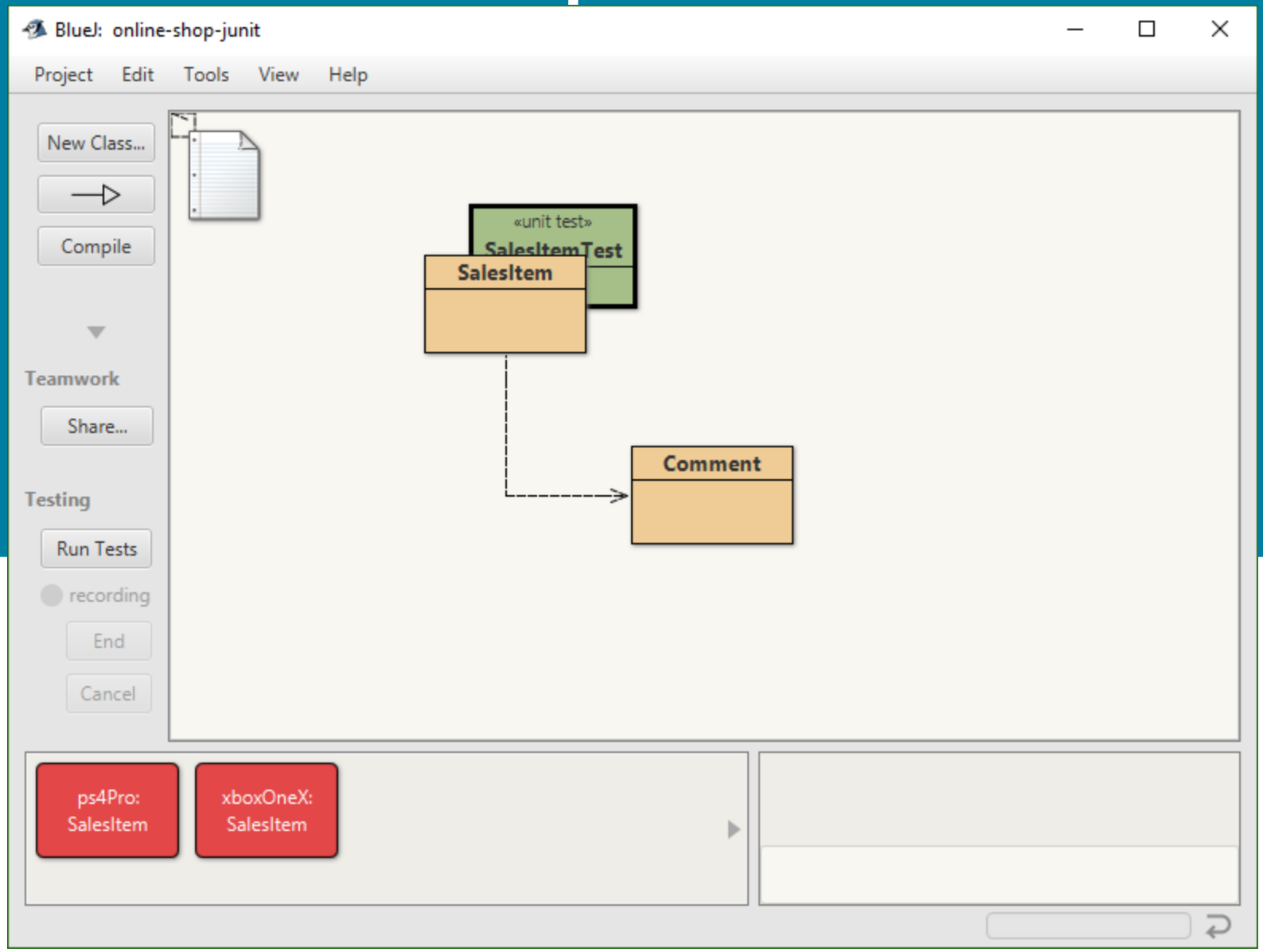
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

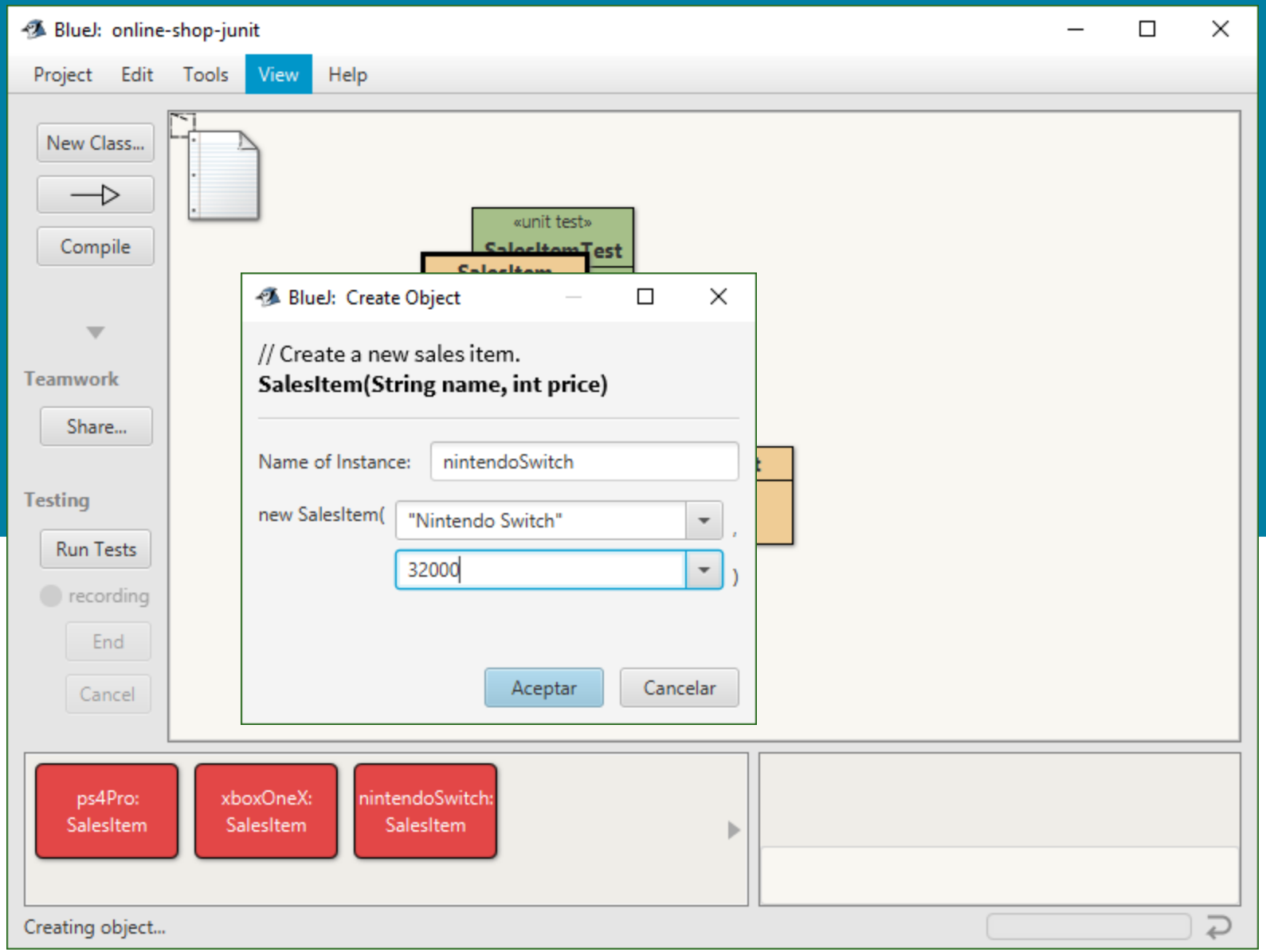
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

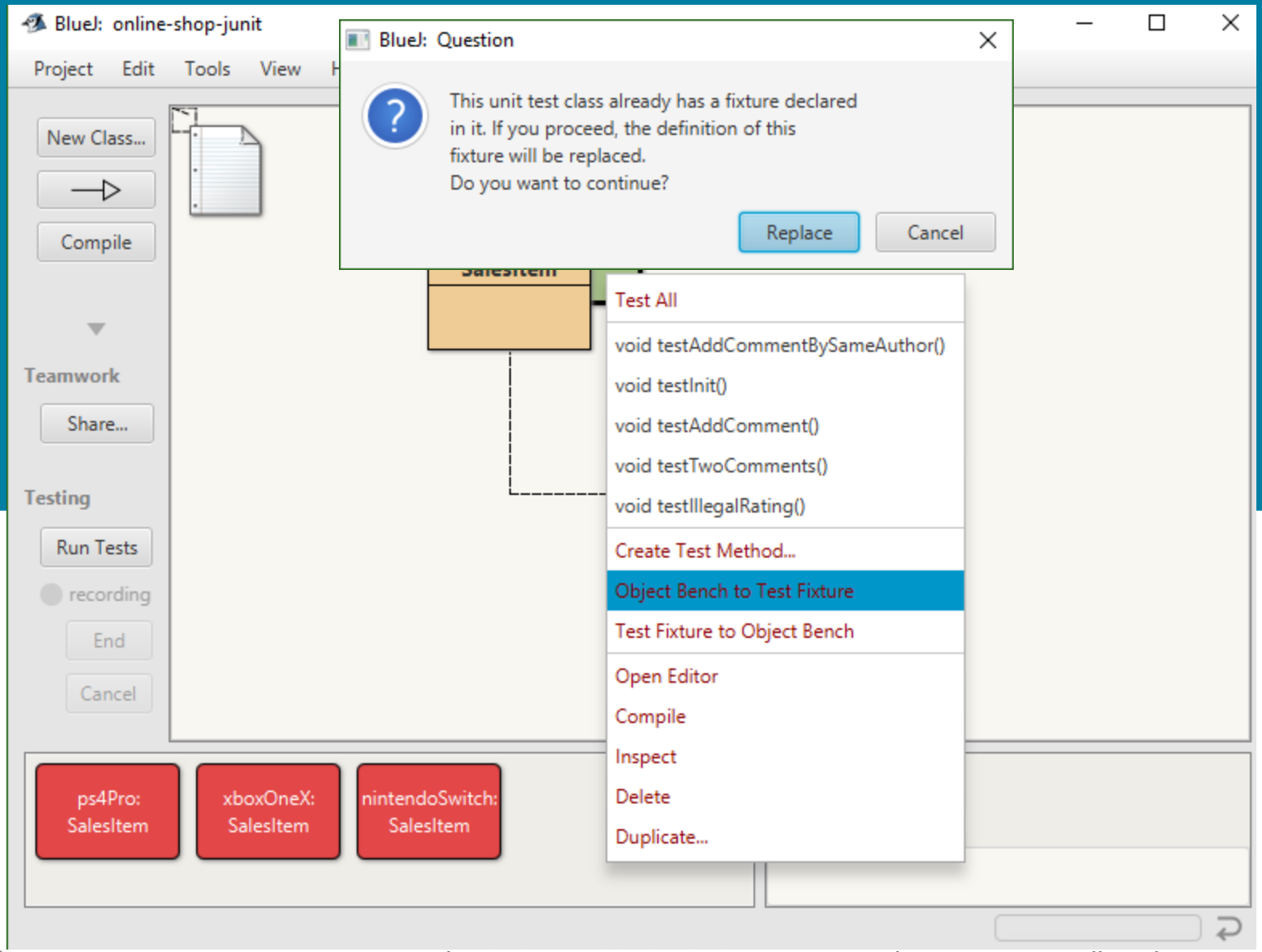
Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

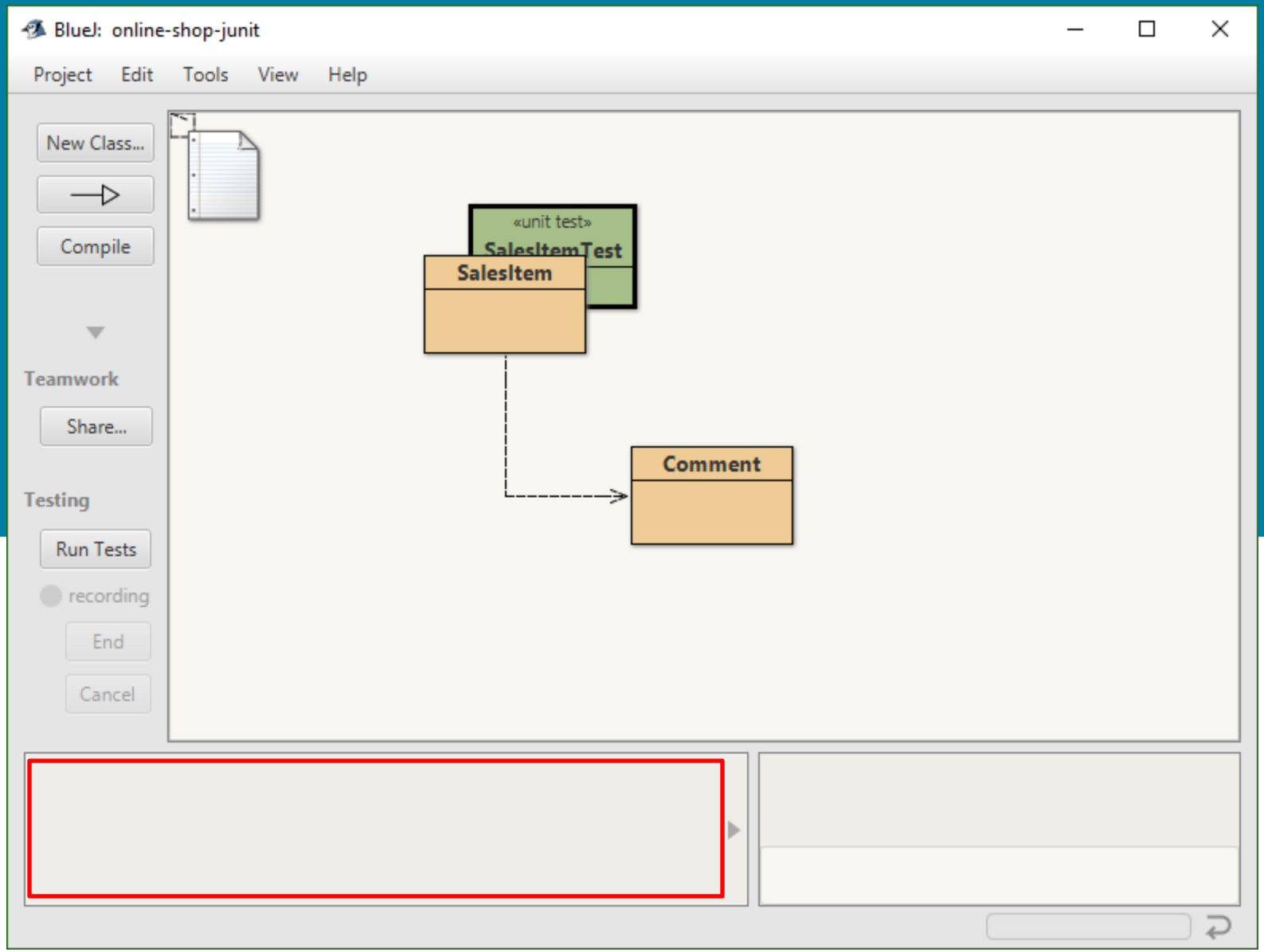
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Online-Shop-Junit – Usando *Fixtures*

SalesItemTest

Compile | Undo | Cut | Copy | Paste | Find... | Close | Source Code

```
7  * The test class SalesItemTest.
8  *
9  * @author  mik
10 * @version 0.1
11 */
12 public class SalesItemTest
13 {
14     private SalesItem ps4Pro;
15     private SalesItem xboxOneX;
16     private SalesItem nintendoSwitch;
17
18
19     /**
20      * Default constructor for test class SalesItemTest
21      */
22     public SalesItemTest()
23     {
24     }
25
26     /**
27      * Sets up the test fixture.
28      *
29      * Called before every test case method.
30      */
31     @Before
32     public void setUp()
33     {
34         ps4Pro = new SalesItem("ps4 Pro 1TB", 40000);
35         xboxOneX = new SalesItem("Xbox One X 1Tb", 45000);
36         nintendoSwitch = new SalesItem("Nintendo Switch", 32000);
37     }
```

Pea served

Eligiendo una estrategia de prueba

- Debemos ser conscientes de las estrategias disponibles.
- Elegir estrategias adecuadas al momento del desarrollo.
- Automatizar siempre que sea posible:
  - Reduce la monotonía.
  - Reduce el error humano.
  - Hace que las (re)pruebas sean más probables.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otra Bibliografía:

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Dr. Becka Morgan- Introduction to Computer Science. Presentations. University of Western Oregon
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved